

Python Practice Workbook

Module 2: Stationary Models

AR, MA, ARMA, ARIMA, SARIMA

Time Series Analytics T6725 — M.Sc. Data Science Sem IV

Symbiosis International (Deemed) University

Primary Dataset: MRF.NS — Supporting: BTC-USD, GC=F

Standard Setup Block — Run this first in every Colab session

```
!pip install yfinance statsmodels arch -q
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

Problem Distribution

Difficulty	Count	Types Used
EASY	10	FIB, FTE, MCQ, PTO, SA
MEDIUM	12	FIB, FTE, GOR, MTF, MCQ, WFS
HARD	12	FIB, FTE, MCQ, WFS, PTO, SA
EXAM	6	Multi-part
TOTAL	40	All 8 types

Section A

Easy Problems — M2.001 to M2.010

M2.001 Fill in the Blank **EASY**

Topic: ADF Unit Root Test — basic usage

Fill in the **four blanks** to run the Augmented Dickey-Fuller test on MRF closing prices and print the result.

```

1 from statsmodels.tsa.stattools import _____ # import
   ADF test
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
   end="2024-12-31")
4 close = mrf["Close"]._____() # remove
   missing values
5
6 # Run ADF test
7 adf_result = adfuller(_____)
8
9 print(f"ADF Statistic : {adf_result[_____]:.4f}")
10 print(f"p-value      : {adf_result[1]:.4f}")
11 print(f"Critical 5%   : {adf_result[4]['5%']:.4f}")
12
13 if adf_result[1] < 0.05:
14     print("STATIONARY -- reject unit root")
15 else:
16     print("NON-STATIONARY -- unit root present")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.stattools import adfuller # adfuller
   is the ADF test function
6
7 mrf = yf.download("MRF.NS", start="2019-01-01",
   end="2024-12-31")
8 close = mrf["Close"].dropna() # dropna()
   removes NaN rows
9
10 # Run ADF test on the raw closing price series
11 adf_result = adfuller(close) # pass the
   Series directly
12

```

```

13 print(f"ADF Statistic : {adf_result[0]:.4f}") # index 0 =
    test statistic
14 print(f"p-value      : {adf_result[1]:.4f}") # index 1 =
    p-value
15 print(f"Critical 5%   : {adf_result[4]['5%']:.4f}") #
    index 4 = critical values dict
16
17 if adf_result[1] < 0.05:
18     print("STATIONARY -- reject unit root")
19 else:
20     print("NON-STATIONARY -- unit root present") # MRF
    price levels show this

```

② Plain English

The ADF test checks whether a series has a unit root — meaning it wanders without a fixed mean. Price levels like MRF's closing price almost always fail this test (high p-value → non-stationary). You must difference the series before fitting ARIMA-family models.

③ Common Mistake

Passing `mrf["Close"]` without `.dropna()` — NaN values cause `adfuller()` to raise a `MissingDataError`. Also, `adf_result[0]` is the test statistic and `adf_result[1]` is the p-value — students frequently swap these indices.

④ Real World Link

Every ARIMA model you build on MRF starts with this test. If $p > 0.05$ on the price level (it will be), you difference once and test again. The number of times you difference to achieve stationarity becomes the d parameter in `ARIMA(p, d, q)`.

M2.002 Fill in the Blank **EASY**

Topic: First differencing to achieve stationarity

Fill in the blanks to difference MRF log prices and confirm stationarity.

```

1 mrf = yf.download("MRF.NS", start="2019-01-01",
    end="2024-12-31")
2 close = mrf["Close"].dropna()
3
4 # Step 1: Take log
5 log_price = np._____(close)
6
7 # Step 2: First difference (log returns)
8 log_returns = log_price._____.dropna()
9
10 # Step 3: ADF test on the differenced series
11 from statsmodels.tsa.stattools import adfuller
12 result = adfuller(_____)
13
14 print(f"p-value after differencing: {result[1]:.6f}")

```

```
15 print("Stationary?", result[1] > 0.05)
```

Solution

① Correct Code

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.stattools import adfuller
7
8 mrf = yf.download("MRF.NS", start="2019-01-01",
9                 end="2024-12-31")
9 close = mrf["Close"].dropna()
10
11 # Step 1: Take log -- stabilises variance
12 log_price = np.log(close)
13
14 # Step 2: First difference = log returns
15 log_returns = log_price.diff().dropna() # .diff()
16                                     subtracts lag-1 from each value
17
18 # Step 3: ADF test on the differenced series
19 result = adfuller(log_returns) # log returns
20                                     should be stationary
21
22 print(f"p-value after differencing: {result[1]:.6f}")
23 print("Stationary?", result[1] < 0.05) # True means
24                                     stationary -- ready to model
```

② Plain English

Raw MRF prices wander (non-stationary, $p > 0.05$ on ADF). After taking log and differencing once, you get log returns — and the ADF p-value drops dramatically (typically < 0.001), confirming stationarity. This one-step differencing corresponds to $d = 1$ in ARIMA(p, 1, q).

③ Common Mistake

Differencing the raw price directly without taking log first — this works for stationarity in mean but doesn't stabilise variance. For financial data, log-differencing (log returns) is the standard because it also handles the multiplicative nature of price changes.

④ Real World Link

Every ARIMA model fitted to MRF returns uses $d = 1$ implicitly — the “I” in ARIMA stands for Integrated, meaning the series was differenced d times. If you're working with already-differenced returns, set $d = 0$. Always confirm with ADF

before choosing d .

M2.003 Multiple Choice **EASY**

Topic: AR(p) — PACF order selection rule

You plot the PACF of MRF log returns and observe significant spikes at lags 1 and 2, with all subsequent lags falling inside the confidence bands. What AR order does this suggest?

- (A) AR(1) — only the first lag matters
- (B) AR(2) — PACF cuts off after lag 2
- (C) MA(2) — use ACF, not PACF, for AR order
- (D) ARMA(2,2) — both ACF and PACF cut off at lag 2

Solution

① **Correct Answer:** (B)

② **Plain English**

For AR(p) identification, the rule is: **PACF cuts off at lag p** . Significant spikes at lags 1 and 2 with clean cutoff after lag 2 $\rightarrow$ AR(2). The PACF measures “pure” correlation at lag k after removing the effect of all intermediate lags — it’s the signature tool for AR order selection.

③ **Common Mistake**

Choosing (C) — a very common reversal. The memory trick: **PACF $\rightarrow$ AR(p)** (both start with letters related to the “autoregressive” side). **ACF $\rightarrow$ MA(q)** (ACF cuts off at lag q for a pure MA process). Get this pair wrong and you’ll misspecify every ARIMA model.

④ **Real World Link**

In practice, MRF return PACF rarely shows clean cutoffs — financial data is messy. But the rule still guides you: look for the last lag where the PACF spike is clearly outside the bands. Then compare AIC/BIC for neighbouring orders.

M2.004 Multiple Choice **EASY**

Topic: MA(q) — ACF order selection rule

The ACF of MRF log returns shows a significant spike only at lag 1, and all other lags are within the confidence bands. What model does this suggest?

- (A) AR(1) — because only one lag matters
- (B) MA(1) — ACF cuts off after lag 1
- (C) ARMA(1,1) — when only one lag is significant
- (D) ARIMA(0,1,0) — one significant lag means $d = 1$

Solution**① Correct Answer: (B)****② Plain English**

For MA(q) identification, the rule is: **ACF cuts off at lag q** . A single significant spike at lag 1 with immediate cutoff $\rightarrow$ MA(1). In an MA(1) process, today's value depends on today's shock plus yesterday's shock — no further memory beyond one period.

③ Common Mistake

Choosing (A) — AR(1) is identified by PACF cutoff at lag 1, not ACF. The ACF of an AR(1) process decays geometrically (exponential tail) rather than cutting off sharply. A sharp cutoff in the ACF is the hallmark of a pure MA process, not AR.

④ Real World Link

If MRF log returns follow an MA(1), today's return is influenced by yesterday's surprise (news shock) but nothing beyond that. A strong earnings announcement that catches the market off-guard creates a one-period ripple — the MA(1) captures exactly this kind of single-lag market reaction.

M2.005 Predict the Output EASY

Topic: ARIMA model order — reading parameters

Without running the code, predict what will be printed.

```

1 from statsmodels.tsa.arima.model import ARIMA
2 import numpy as np
3
4 np.random.seed(0)
5 y = np.cumsum(np.random.randn(200)) # random walk
6
7 model = ARIMA(y, order=(1, 1, 0))
8 fit    = model.fit()
9
10 print(f"AR order : {fit.model.order[0]}")
11 print(f"I  order : {fit.model.order[1]}")
12 print(f"MA order : {fit.model.order[2]}")
13 print(f"AIC      : {fit.aic:.2f}")
14 print(f"Converged: {fit.mle_retvals['converged']}")

```

Write the predicted output for the first three lines. For AIC, state whether it will be positive or negative and why. For Converged, predict True or False.

Solution**① Predicted Output**

```
AR order : 1
I order : 1
MA order : 0
AIC : [large number, positive or negative, typically -200 to
+600 for n=200]
Converged: True
```

② Plain English

The first three lines are deterministic — they echo back the `order=(1,1,0)` tuple you passed in. `fit.model.order` stores (p, d, q) exactly as specified. AIC can be positive or negative depending on the data scale and model likelihood. `converged=True` means the optimizer found a solution (very common for simple models).

③ Common Mistake

Expecting `fit.model.order` to return the *estimated* orders — it doesn't. It returns the *specified* orders. There is no automatic order selection inside `ARIMA()` — you pass what you want. Auto-selection requires `auto_arima` from the `pmdarima` library.

④ Real World Link

When you write a model report for MRF analysis, you always document the (p, d, q) order alongside the AIC. “ARIMA(1,1,0) with AIC = 412.3” is a complete, reproducible model specification. Another analyst can reconstruct your exact model from this description.

M2.006 Short Answer EASY

Topic: Stationarity — definition

In 2 sentences, define a weakly stationary time series. State one property of MRF's daily closing price that makes it non-stationary.

Solution**① Model Answer**

A weakly stationary (covariance-stationary) time series has three properties: constant mean, constant variance, and autocovariance that depends only on the lag between observations — not on when in time the observations are taken. MRF's daily closing price is non-stationary because its mean is not constant — the price trended from approximately 55,000 in 2019 to over 1,20,000 by 2024, violating the constant-mean requirement.

② Common Mistake

Saying “the price changes every day” — that is not the definition of non-stationarity. The issue is that the *statistical properties* (mean, variance) change over time. A series can fluctuate heavily and still be stationary if it fluctuates around a fixed mean with stable variance.

③ Real World Link

The entire ARIMA framework is built for stationary series. Before you fit any model to MRF, you must transform it into a stationary series — typically by taking log returns (log + first difference). The *d* parameter in ARIMA records how many times you differenced.

M2.007 Find the Error **EASY**

Topic: ARIMA fitting — wrong order format
This code crashes. Find and fix the error.

```
1 from statsmodels.tsa.arima.model import ARIMA
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
4                   end="2024-12-31")
5 returns = np.log(mrf["Close"]).diff().dropna()
6
7 # Fit ARIMA
8 model = ARIMA(returns, order=[1, 0, 1]) # ERROR
9 fit = model.fit()
10 print(fit.summary())
```

Solution

① Correct Code

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.arima.model import ARIMA
7
8 mrf = yf.download("MRF.NS", start="2019-01-01",
9                   end="2024-12-31")
10 returns = np.log(mrf["Close"]).diff().dropna()
11
12 # FIX: order must be a TUPLE (p, d, q), not a list
13 model = ARIMA(returns, order=(1, 0, 1)) # parentheses,
14 fit = model.fit()
15 print(fit.summary())
```


② Plain English

The `ARIMA()` function expects `order` as a **tuple** (`p`, `d`, `q`), not a list [`p`, `d`, `q`]. While Python lists and tuples look similar, `statsmodels` performs type checking internally and raises a `ValueError` when a list is passed for the `order` parameter. Always use round brackets `()` for order specification.

③ Common Mistake

Thinking lists and tuples are interchangeable in all Python contexts. Many scientific libraries (`statsmodels`, `scikit-learn`) explicitly require tuples for parameter specifications, especially when those parameters define model structure. When in doubt, use `()`.

④ Real World Link

This kind of type error is frustrating in practice because the error message often points to an internal library line rather than your own code. Knowing the correct syntax for `ARIMA(order=(p,d,q))` from memory prevents this class of error entirely.

M2.008 Fill in the Blank **EASY**

Topic: Fitting AR(1) and reading coefficients

Fill in the blanks to fit an AR(1) model to MRF returns and print the AR coefficient.

```
1 from statsmodels.tsa.arima.model import ARIMA
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
4     end="2024-12-31")
5 returns = np.log(mrf["_____"]).diff().dropna()
6
7 # Fit AR(1): p=1, d=0, q=0
8 model = ARIMA(returns, order=(_____, 0, _____))
9 fit = model._____()
10
11 # Print the AR(1) coefficient
12 ar_coef = fit.params["_____"]
13 print(f"AR(1) coefficient: {ar_coef:.6f}")
14 print(f"AIC: {fit._____.2f}")
```

Solution

① Correct Code

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.arima.model import ARIMA
```

```

7
8 mrf = yf.download("MRF.NS", start="2019-01-01",
9                   end="2024-12-31")
10 returns = np.log(mrf["Close"]).diff().dropna() # "Close"
11 # Fit AR(1): p=1, d=0, q=0
12 model = ARIMA(returns, order=(1, 0, 0)) # AR
13 fit = model.fit() # .fit()
14 # Print the AR(1) coefficient
15 ar_coef = fit.params["ar.L1"] # statsmodels names AR
16 # lag-1 as "ar.L1"
17 print(f"AR(1) coefficient: {ar_coef:.6f}")
18 print(f"AIC: {fit.aic:.2f}") # .aic attribute, not a
19 # method

```

② Plain English

An AR(1) model says today's return equals a constant plus φ_1 times yesterday's return plus noise. The coefficient `ar.L1` is that φ_1 value. For MRF returns, this will be a small number (close to zero) — strong evidence that past returns have minimal predictive power over future returns.

③ Common Mistake

Using `fit.params["ar1"]` or `fit.params["AR1"]` — the exact parameter name is `"ar.L1"` in statsmodels (lowercase “ar”, dot, capital “L”, number). To avoid guessing, always print `fit.params` first to see all parameter names before accessing individual ones.

④ Real World Link

If the AR(1) coefficient for MRF returns is close to zero and statistically insignificant, that's evidence supporting market efficiency — yesterday's return doesn't help predict today's. This is the most common finding for liquid, large-cap stocks like MRF.

M2.009 Multiple Choice **EASY**

Topic: ARIMA(p, d, q) — role of d

In ARIMA(2, 1, 1) fitted to MRF, what does $d = 1$ mean?

- (A) The model uses 1 lag for the autoregressive component
- (B) The series was differenced once before fitting AR and MA components
- (C) The seasonal period is 1 year
- (D) The model has 1 moving average term at lag 1

Solution**① Correct Answer: (B)****② Plain English**

In ARIMA(p, d, q): p = AR order (how many lagged values), d = degree of differencing (how many times you subtracted consecutive values), q = MA order (how many lagged errors). $d=1$ means the raw series was differenced once — i.e., we modelled first differences (returns) rather than the price level itself.

③ Common Mistake

Confusing d with the AR lag count. The AR lags are counted by p . The d parameter only controls differencing — it's a preprocessing step baked into the model specification. When $d = 1$, ARIMA internally differences your series before fitting the ARMA(p, q) part.

④ Real World Link

For MRF prices, $d = 1$ is almost always the right choice. $d = 2$ (double differencing) is rarely needed for financial price series — it would imply that even first differences (returns) are non-stationary. If ADF on returns already shows stationarity, use $d = 1$ total (applied to prices) or $d = 0$ (if you pass returns directly).

M2.010 Short Answer EASY

Topic: White noise vs stationary

In 2 sentences, explain the difference between a white noise process and a stationary process. Is every stationary process a white noise process?

Solution**① Model Answer**

A stationary process has constant mean, constant variance, and autocovariance that depends only on lag — but it can still have autocorrelation structure (like AR or MA processes). A white noise process is a *special case* of stationarity: it is stationary AND has zero autocorrelation at all lags — so every white noise process is stationary, but not every stationary process is white noise. An AR(1) is stationary but has non-zero autocorrelation.

② Common Mistake

Saying stationarity and white noise are the same thing. They're not — white noise is a strict subset. An AR(1) process with $|\varphi| < 1$ is stationary but is clearly not white noise (it has a decaying ACF). Many students first encounter “stationary” and “white noise” in the same lecture and conflate them.

③ Real World Link

MRF log returns that pass the ADF test are stationary — but they may still have AR or MA structure worth modelling (check ACF/PACF). Only after you fit an ARIMA model and verify that the residuals are white noise (Ljung-Box test on residuals) have you successfully removed all predictable structure.

Section B

Medium Problems — M2.011 to M2.022

M2.011 *Fill in the Blank* **MEDIUM****Topic:** ACF and PACF for model identification

Fill in the blanks to plot ACF and PACF side-by-side for MRF returns.

```

1 from statsmodels.graphics.tsaplots import plot_acf, _____
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
4                   end="2024-12-31")
5 returns = np.log(mrf["Close"]).diff()._____()
6
7 fig, axes = plt.subplots(1, 2, figsize=(_____, 4))
8
9 # ACF -- use to identify MA(q) order
10 plot_acf(returns, lags=30, alpha=_____, ax=axes[0],
11          zero=False)
12 axes[0].set_title("ACF -- MA order identification")
13
14 # PACF -- use to identify AR(p) order
15 _____(returns, lags=30, alpha=0.05, ax=axes[_____,],
16            zero=False)
17 axes[1].set_title("PACF -- AR order identification")
18
19 plt.tight_layout()
20 plt.show()

```

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import warnings
6 warnings.filterwarnings('ignore')
7 from statsmodels.graphics.tsaplots import plot_acf,
8   plot_pacf
9
10 mrf = yf.download("MRF.NS", start="2019-01-01",
11                  end="2024-12-31")
12 returns = np.log(mrf["Close"]).diff().dropna()
13
14 fig, axes = plt.subplots(1, 2, figsize=(14, 4))
15
16 # ACF -- use to identify MA(q) order

```

```

15 plot_acf(returns, lags=30, alpha=0.05, ax=axes[0],
16         zero=False)
17
18 # PACF -- use to identify AR(p) order
19 plot_pacf(returns, lags=30, alpha=0.05, ax=axes[1],
20          zero=False)
21 axes[1].set_title("PACF -- AR order identification")
22
23 plt.tight_layout()
24 plt.show()
25 # Rule: ACF cuts off at q -> MA(q). PACF cuts off at p ->
26       AR(p).

```

② Plain English

The ACF-PACF pair is your first diagnostic tool for ARIMA order selection. Read them together: ACF tells you the MA order (q = last significant lag before cutoff), PACF tells you the AR order (p = last significant lag before cutoff). If both decay slowly, you have an ARMA process and need to try combinations.

③ Common Mistake

Setting `zero=False` is important but often forgotten — by default, lag 0 is plotted with ACF=1 (perfect self-correlation), which compresses the scale and makes other spikes hard to read. `zero=False` removes this trivial lag and gives a cleaner, more readable plot.

④ Real World Link

For MRF returns, the ACF and PACF will likely show very few significant spikes — consistent with near-efficient market behavior. This tells you a simple ARIMA(0,1,0) or ARIMA(1,1,0) may be sufficient, and there's no need to over-engineer a high-order model.

M2.012 Find the Error **MEDIUM**

Topic: SARIMA seasonal order specification

This code runs but produces a **non-seasonal model** when a seasonal model was intended. Find the conceptual error.

```

1 from statsmodels.tsa.statespace.sarimax import SARIMAX
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
4                 end="2024-12-31")
5 close = mrf["Close"].dropna()
6
7 # Attempt: SARIMA with weekly seasonality on daily data
8 model = SARIMAX(close,
9                 order=(1, 1, 1),
10                 seasonal_order=(1, 1, 1, 0)) # ERROR:
11                                             wrong seasonal period

```

```

10 fit = model.fit(dis=False)
11 print(fit.summary())

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.statespace.sarimax import SARIMAX
6
7 mrf = yf.download("MRF.NS", start="2019-01-01",
8                 end="2024-12-31")
9 close = mrf["Close"].dropna()
10
11 # FIX: seasonal_order is (P, D, Q, s) where s = periods
12 #      per season
13 # s=0 means NO seasonality -- not a seasonal model at all
14 # For daily data with weekly seasonality: s=5 (5 trading
15 #      days per week)
16 # For daily data with annual seasonality: s=252 (approx
17 #      trading days/year)
18
19 model = SARIMAX(close,
20                 order=(1, 1, 1),
21                 seasonal_order=(1, 1, 1, 5)) # FIX: s=5
22                                             # for weekly season
23 fit = model.fit(dis=False)
24 print(fit.summary())

```

② Plain English

The `seasonal_order` parameter is (P, D, Q, s) where s is the seasonal period — the number of time steps in one complete seasonal cycle. Setting `s=0` tells the model there is no seasonal structure, making it identical to a plain ARIMA. For weekly patterns in daily trading data, use `s=5`.

③ Common Mistake

Confusing the four positions in `seasonal_order`. Always ask: “How many rows of my data make one seasonal cycle?” Daily data with annual seasonality → `s = 252`. Monthly data with annual seasonality → `s = 12`.

④ Real World Link

MRF trading volumes and return volatility often show day-of-week effects. A SARIMA with `s=5` attempts to capture this weekly pattern. Whether it genuinely improves forecast accuracy over plain ARIMA must be verified by comparing AIC values.

M2.013 *Get the Order Right* **MEDIUM****Topic:** Box-Jenkins ARIMA model building workflowThese **7 steps** of the Box-Jenkins methodology are scrambled. Write the correct order.

- (A) Check residuals with Ljung-Box test — are they white noise?
- (B) Plot ACF and PACF of the stationary series — identify p and q candidates
- (C) Run ADF test on the raw series — is it stationary?
- (D) Use the fitted model to generate forecasts
- (E) Fit ARIMA(p, d, q) using the identified orders
- (F) If not stationary — difference the series until ADF $p < 0.05$
- (G) Compare multiple candidate models using AIC and BIC — select the best

Solution**① Correct Order:** C → F → B → E → G → A → D

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, warnings
3 warnings.filterwarnings('ignore')
4 from statsmodels.tsa.stattools import adfuller
5 from statsmodels.graphics.tsaplots import plot_acf,
   plot_pacf
6 from statsmodels.tsa.arima.model import ARIMA
7 from statsmodels.stats.diagnostic import acorr_ljungbox
8 import matplotlib.pyplot as plt
9
10 mrf = yf.download("MRF.NS", start="2019-01-01",
   end="2024-12-31")
11 close = mrf["Close"].dropna()
12
13 # STEP C: Test stationarity of raw series
14 adf_raw = adfuller(close)
15 print(f"Step C -- ADF on price: p = {adf_raw[1]:.4f}") #
   expect p > 0.05
16
17 # STEP F: Difference until stationary
18 returns = np.log(close).diff().dropna()
19 adf_diff = adfuller(returns)
20 print(f"Step F -- ADF on returns: p = {adf_diff[1]:.6f}")
   # expect p < 0.05 -> d=1
21
22 # STEP B: Plot ACF / PACF to identify p and q
23 fig, ax = plt.subplots(1, 2, figsize=(14, 4))
24 plot_acf(returns, lags=20, ax=ax[0], zero=False)
25 plot_pacf(returns, lags=20, ax=ax[1], zero=False)
26 plt.tight_layout(); plt.show()

```

```

27
28 # STEP E: Fit candidate models based on inspection
29 fit_101 = ARIMA(returns, order=(1, 0, 1)).fit()
30 fit_100 = ARIMA(returns, order=(1, 0, 0)).fit()
31
32 # STEP G: Compare AIC
33 print(f"Step G -- AIC ARMA(1,1): {fit_101.aic:.2f}")
34 print(f"Step G -- AIC AR(1): {fit_100.aic:.2f}")
35 best = fit_101 if fit_101.aic < fit_100.aic else fit_100
36
37 # STEP A: Residual diagnostics on best model
38 lb = acorr_ljungbox(best.resid, lags=10, return_df=True)
39 print(f"Step A -- Min residual p-value:
40       {lb['lb_pvalue'].min():.4f}")
41
42 # STEP D: Forecast
43 forecast = best.forecast(steps=10)
44 print(f"Step D -- 10-day forecast:\n{forecast}")

```

② Plain English

Box-Jenkins is a disciplined 7-step recipe: confirm the problem (non-stationarity), fix it (difference), read the ACF/PACF diagnostic, fit, compare, validate, forecast. Skipping any step — especially the residual check (Step A) — means your model might look good but is internally misspecified.

③ Common Mistake

Going directly from Step C to Step E (skipping differencing and ACF/PACF). This gives you a model that may technically fit but produces biased parameter estimates and unreliable forecasts. The workflow is not just a suggestion — each step guards against a specific class of modelling error.

④ Real World Link

Every professional time series report for MRF follows this structure: test → transform → identify → fit → validate → forecast. If a financial analyst hands you an ARIMA model without showing you the ACF/PACF plots and residual diagnostics, you should ask for them.

M2.014 Match the Following **MEDIUM**

Topic: Model identification rules — ACF/PACF patterns

Match each **pattern** (Column A) to the **correct model** (Column B).

#	Column A — Pattern	#	Column B — Model
1	ACF decays geometrically; PACF cuts off at lag 2	(a)	MA(2)
2	ACF cuts off at lag 1; PACF decays geometrically	(b)	AR(2)
3	Both ACF and PACF decay slowly	(c)	ARMA(p,q)
4	ACF cuts off at lag 2; PACF decays geometrically	(d)	White noise
5	Both ACF and PACF: all lags within bands	(e)	MA(1)

Solution

① Correct Matches

Pattern	Match
1. ACF decays geometrically; PACF cuts off at lag 2	(b) AR(2)
2. ACF cuts off at lag 1; PACF decays geometrically	(e) MA(1)
3. Both ACF and PACF decay slowly	(c) ARMA(p,q)
4. ACF cuts off at lag 2; PACF decays geometrically	(a) MA(2)
5. Both within bands	(d) White noise

② Plain English

The identification rules form two symmetric pairs: **AR uses PACF**, **MA uses ACF**. When both decay, you have ARMA and need to try combinations. When both are clean (all within bands), you have white noise and no ARMA structure is needed.

Summary: AR(p) → PACF cuts at lag p — MA(q) → ACF cuts at lag q — ARMA → both decay — White noise → both inside bands.

③ Common Mistake

Thinking geometric decay = “many significant spikes.” Geometric decay means each spike is slightly smaller than the previous one — it never sharply cuts off like an MA pattern does. Learning to visually distinguish “gradual geometric decay” from “sharp cutoff” is the core skill of ACF/PACF reading.

④ Real World Link

For MRF returns, you’ll likely see pattern 5 (near-white noise) — most spikes inside bands, at most 1–2 marginally significant lags. This tells you a low-order ARMA model is appropriate and high-order models are likely overfitting.

M2.015 Fill in the Blank MEDIUM

Topic: AIC and BIC for model comparison

Fill in the blanks to fit three ARIMA candidates on MRF returns and select the best by AIC.

```

1 from statsmodels.tsa.arima.model import ARIMA
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
4                 end="2024-12-31")
5 returns = np.log(mrf["Close"]).diff().dropna()
6
7 candidates = [(1,0), (1,1), (2,1)]
8 results = {}
9
10 for (p, q) in candidates:
11     model = ARIMA(returns, order=(p, _____, q)) # d=0,
12     returns stationary
13     fit = model._____()
14     results[(p,q)] = {"AIC": fit._____, "BIC": fit.bic}
15     print(f"ARMA({p},{q}) -> AIC={fit.aic:.2f},
16           BIC={fit.bic:.2f}")
17
18 # Select model with lowest AIC
19 best_order = min(results, key=lambda x:
20                 results[x][_____])
21 print(f"\nBest model by AIC: ARMA{best_order}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.arima.model import ARIMA
7
8 mrf = yf.download("MRF.NS", start="2019-01-01",
9                 end="2024-12-31")
10 returns = np.log(mrf["Close"]).diff().dropna()
11
12 candidates = [(1,0), (1,1), (2,1)]
13 results = {}
14
15 for (p, q) in candidates:
16     model = ARIMA(returns, order=(p, 0, q)) # d=0:
17     returns already stationary
18     fit = model.fit()
19     results[(p,q)] = {"AIC": fit.aic, "BIC": fit.bic}
20     print(f"ARMA({p},{q}) -> AIC={fit.aic:.2f},
21           BIC={fit.bic:.2f}")
22
23

```

```

20 # Select model with lowest AIC
21 best_order = min(results, key=lambda x: results[x]["AIC"])
    # "AIC" matches dict key
22 print(f"\nBest model by AIC: ARMA{best_order}")

```

② Plain English

AIC (Akaike Information Criterion) penalises model complexity while rewarding fit quality — lower is better. By looping over candidate (p,q) pairs and comparing AIC, you find the model that explains the data well without unnecessary parameters. BIC applies a stronger penalty for complexity and tends to select simpler models than AIC.

③ Common Mistake

Setting `d=1` when passing returns (already differenced) to ARIMA — this would double-difference your data, producing second differences which are usually non-stationary. Pass `d=0` when your input is already returns; use `d=1` only when your input is the raw price level.

④ Real World Link

In a real MRF analysis report, you'd typically show a table of 6–9 candidate models with their AIC/BIC values, then select the one with the lowest information criterion. This transparent model selection process is expected in academic submissions and is standard practice in quantitative finance research.

M2.016 Find the Error MEDIUM

Topic: SARIMA — confusion between order and seasonal_order

This code runs but the model specification is completely wrong. Find the **two** errors.

```

1 from statsmodels.tsa.statespace.sarimax import SARIMAX
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
    end="2024-12-31")
4 returns = np.log(mrf["Close"]).diff().dropna()
5
6 # Intended: SARIMA(1,0,1)(1,0,1)[5] on MRF returns
7 model = SARIMAX(returns,
8                 order=(1, 0, 1, 5),           # ERROR 1
9                 seasonal_order=(1, 0, 1))     # ERROR 2
10 fit = model.fit(dispatch=False)
11 print(fit.summary())

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q

```

```

2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.statespace.sarimax import SARIMAX
7
8 mrf = yf.download("MRF.NS", start="2019-01-01",
9                 end="2024-12-31")
10 returns = np.log(mrf["Close"]).diff().dropna()
11
12 # FIX 1: order must be a 3-tuple (p, d, q) -- NOT 4
13         elements
14 # FIX 2: seasonal_order must be a 4-tuple (P, D, Q, s) --
15         s is REQUIRED
16
17 model = SARIMAX(returns,
18                 order=(1, 0, 1),           # FIX 1:
19                                     exactly 3 elements
20                 seasonal_order=(1, 0, 1, 5)) # FIX 2: 4
21                                     elements, s=5 for weekly
22 fit = model.fit(dispatch=False)
23 print(fit.summary())

```

② Plain English

Two structural errors. First: `order` must have exactly 3 elements (`p, d, q`) — adding a 4th crashes the model. Second: `seasonal_order` must have exactly 4 elements (`P, D, Q, s`) — the seasonal period `s` is not optional. Leaving it out makes `statsmodels` interpret the tuple incorrectly and raises a `ValueError`.

③ Common Mistake

Students who learn SARIMA from textbooks often see the notation `SARIMA(p,d,q)(P,D,Q)[s]` and don't realise that `s` is passed inside the `seasonal_order` tuple, not as a separate argument. There is no `s=` keyword — the period is always the 4th element of `seasonal_order`.

④ Real World Link

In practice, SARIMA models are harder to fit than ARIMA — more parameters, more ways to misspecify. Always print `fit.summary()` and verify the model equation shown at the top matches what you intended. The summary explicitly shows the SARIMAX order, so you can catch specification errors before trusting the output.

M2.017 Multiple Choice MEDIUM

Topic: Stationarity conditions for AR(1)

An AR(1) process is defined as $X_t = \varphi \cdot X_{t-1} + \varepsilon_t$. Which condition on φ guarantees weak stationarity?

- (A) $\varphi > 0$ — the process must be positively autocorrelated
- (B) $|\varphi| < 1$ — the absolute value of φ must be strictly less than 1

- (C) $\varphi = 1$ — this is the boundary case (random walk) which is stationary
 (D) $\varphi > 1$ — explosive processes are eventually mean-reverting

Solution

① Correct Answer: (B)

② Plain English

For AR(1) stationarity, the coefficient must satisfy $|\varphi| < 1$. When $|\varphi| < 1$, shocks to the system decay exponentially over time and the series reverts to its mean — it's stable. When $|\varphi| = 1$, you get a random walk (unit root, non-stationary). When $|\varphi| > 1$, the process explodes — it diverges to $\pm\infty$.

③ Common Mistake

Choosing (C) — $\varphi = 1$ is the **unit root** case, which defines a **non-stationary** random walk. This is the exact case the ADF test is trying to detect and reject. The phrase “unit root” comes from the fact that the characteristic root of the AR polynomial equals 1 when $\varphi = 1$.

④ Real World Link

The ADF test for MRF prices is essentially checking whether $\varphi = 1$ (random walk, non-stationary) or $|\varphi| < 1$ (mean-reverting, stationary). Stock prices almost always behave like random walks ($\varphi \approx 1$) — but returns (first differences) have $|\varphi| \ll 1$ and pass the stationarity test.

M2.018 Write from Scratch MEDIUM

Topic: Fitting ARIMA with residual diagnostics

Write complete code that: (1) downloads MRF data from 2019, (2) computes log returns, (3) fits ARIMA(1,0,1) to the returns, (4) prints the model summary, (5) plots the residuals, (6) runs Ljung-Box test on residuals (20 lags) and prints conclusion.

Solution

① Correct Code

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import warnings
6 warnings.filterwarnings('ignore')
7 from statsmodels.tsa.arima.model import ARIMA
8 from statsmodels.stats.diagnostic import acorr_ljungbox
9
10 # Step 1: Download MRF data
11 mrf = yf.download("MRF.NS", start="2019-01-01",
    end="2024-12-31")
```

```

12
13 # Step 2: Compute log returns
14 returns = np.log(mrf["Close"]).diff().dropna()
15
16 # Step 3: Fit ARIMA(1,0,1) -- ARMA(1,1) on returns
17 model = ARIMA(returns, order=(1, 0, 1))
18 fit = model.fit()
19
20 # Step 4: Print model summary
21 print(fit.summary())
22
23 # Step 5: Plot residuals
24 fig, axes = plt.subplots(2, 1, figsize=(14, 6))
25
26 axes[0].plot(fit.resid, color="steelblue", linewidth=0.8,
27             alpha=0.8)
28 axes[0].axhline(0, color="red", linewidth=1,
29               linestyle="--")
30 axes[0].set_title("ARIMA(1,0,1) Residuals -- MRF Log
31               Returns")
32 axes[0].set_ylabel("Residual")
33
34 axes[1].hist(fit.resid, bins=60, color="steelblue",
35             alpha=0.6, density=True)
36 axes[1].set_title("Residual Distribution")
37 axes[1].set_xlabel("Residual Value")
38 plt.tight_layout()
39 plt.show()
40
41 # Step 6: Ljung-Box test on residuals
42 lb = acorr_ljungbox(fit.resid, lags=20, return_df=True)
43 min_pval = lb["lb_pvalue"].min()
44
45 print(f"\nLjung-Box on Residuals (20 lags)")
46 print(f"Minimum p-value: {min_pval:.4f}")
47
48 if min_pval > 0.05:
49     print("Residuals consistent with white noise -- model
50           is well-specified")
51 else:
52     print("Residuals show autocorrelation -- model needs
53           revision")
54     print("    -> Try higher AR or MA order, or check for
55           seasonal effects")

```

② Plain English

Residual diagnostics are the final exam your model must pass. After fitting ARIMA(1,0,1), you check whether what's left over (residuals) is pure white noise. If Ljung-Box shows $p > 0.05$ across all lags, your model captured all systematic

structure. If not, you've left predictable patterns on the table.

③ Common Mistake

Skipping the residual check because “the model fitted successfully.” A model always fits — the question is whether it fits *correctly*. Significant residual autocorrelation means your chosen (p,q) orders were insufficient and you need to increase one of them.

④ Real World Link

In a trading strategy context, residuals from a correctly specified MRF ARIMA model should look like random noise. If they don't, someone with a better model is exploiting that leftover structure. The Ljung-Box test is your check against leaving money on the table.

M2.019 Fill in the Blank MEDIUM

Topic: SARIMA for MRF with annual seasonality

Fill in the blanks to fit a SARIMA model that captures annual seasonal patterns on monthly MRF data.

```

1 from statsmodels.tsa.statespace.sarimax import SARIMAX
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
4                   end="2024-12-31")
5 close = mrf["Close"].dropna()
6
7 # Resample to MONTHLY data
8 monthly = close.resample("____").last()
9
10 # SARIMA(1,1,1)(1,1,1)[12]
11 model = SARIMAX(monthly,
12                 order=(1, _____, 1),
13                 seasonal_order=(_____, 1, _____, _____),
14                 enforce_stationarity=False,
15                 enforce_invertibility=False)
16
17 fit = model.fit(_____) # suppress optimisation output
18 print(fit.summary())

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.statespace.sarimax import SARIMAX
6

```

```

7 mrf = yf.download("MRF.NS", start="2019-01-01",
8   end="2024-12-31")
9
10 # Resample to MONTHLY data -- last trading day of each
11   month
12
13 monthly = close.resample("ME").last() # "ME" = Month End
14
15 # SARIMA(1,1,1)(1,1,1)[12]
16 model = SARIMAX(monthly,
17   order=(1, 1, 1), #
18   non_seasonal: AR=1, diff=1, MA=1
19   seasonal_order=(1, 1, 1, 12), # seasonal:
20   P=1, D=1, Q=1, s=12
21   enforce_stationarity=False,
22   enforce_invertibility=False)
23
24 fit = model.fit(dispatch=False) # dispatch=False suppresses
25   optimiser output
26 print(fit.summary())

```

② Plain English

SARIMA extends ARIMA with an extra set of seasonal AR, differencing, and MA terms. The `s=12` tells the model to look for patterns that repeat every 12 months. Seasonal differencing (`D=1`) removes annual trends — the equivalent of differencing at lag 12 rather than lag 1.

③ Common Mistake

Using `dispatch=True` (the default) during development — this floods your output with hundreds of optimizer iteration lines. Always set `dispatch=False` when fitting SARIMA interactively. You'll still see the summary; you just avoid the noise.

④ Real World Link

MRF's tyre replacement demand peaks in summer (April–June in India) before monsoon season when road conditions deteriorate. A `SARIMA(1,1,1)(1,1,1)[12]` on monthly prices attempts to model this predictable annual rhythm alongside the underlying trend.

M2.020 Write from Scratch MEDIUM

Topic: ADF test — automated differencing loop

Write code that: (1) downloads MRF closing prices, (2) runs ADF test on the raw series, (3) if non-stationary: differences once and re-tests, (4) if still non-stationary: differences again and re-tests, (5) reports how many differences were needed (`d` value) and the final p-value.

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.stattools import adfuller
7
8 # Step 1: Download MRF
9 mrf = yf.download("MRF.NS", start="2019-01-01",
10                  end="2024-12-31")
11 series = np.log(mrf["Close"]).dropna() # work in
12                                         log-space throughout
13
14 d = 0 # differencing counter
15 max_d = 3 # never difference more than 3 times in
16           practice
17
18 print("="*55)
19 print(" Automated Differencing to Achieve Stationarity")
20 print("="*55)
21
22 while d <= max_d:
23     result = adfuller(series)
24     pval = result[1]
25     stat = result[0]
26
27     print(f"\nd = {d}")
28     print(f" ADF Statistic : {stat:.4f}")
29     print(f" p-value : {pval:.6f}")
30
31     if pval < 0.05:
32         print(f" STATIONARY at d={d} (p={pval:.4f} <
33             0.05)")
34         print(f"\n-> Recommended ARIMA d parameter: {d}")
35         break
36     else:
37         print(f" Non-stationary (p={pval:.4f} >= 0.05) --
38             differencing...")
39         series = series.diff().dropna() # first
40                                         difference of current series
41         d += 1
42
43 if d > max_d:
44     print(f"\nWarning: Could not achieve stationarity in
45         {max_d} differences.")

```

② Plain English

This loop automates the “how many times do I difference?” question. It starts on raw log-prices ($d = 0$), runs ADF, and increments d until stationarity is achieved. For MRF, $d = 0$ will fail (price levels non-stationary), $d = 1$ will pass (log returns are stationary), and the loop exits with $d = 1$ as the recommended parameter.

③ Common Mistake

Differencing in a loop without logging first — if the variance of price levels is not stabilised by log transformation, even after achieving mean stationarity, your series may still have heteroscedastic residuals. Working in log-space first gives cleaner results throughout.

④ Real World Link

This kind of automated differencing scan is the manual precursor to `auto.arima()` from `pmdarima`. Understanding the loop helps you appreciate what `auto.arima` does internally — it’s this same systematic test-and-difference procedure, just automated over a grid of (p,d,q) combinations.

M2.021 Multiple Choice MEDIUM

Topic: Information criteria — AIC vs BIC trade-off

You are comparing two ARIMA models for MRF returns:

- **Model A:** ARIMA(2,0,2) — AIC = -3241.5 , BIC = -3218.2
- **Model B:** ARIMA(1,0,1) — AIC = -3238.7 , BIC = -3224.6

Which statement is correct?

- (A) Model A is better by both AIC and BIC — always use Model A
- (B) AIC favours Model A; BIC favours Model B — the right choice depends on your priority
- (C) Model B is better by both criteria — always use Model B
- (D) They are equivalent — the difference is too small to matter

Solution

① Correct Answer: (B)

② Plain English

AIC = -3241.5 (Model A) < -3238.7 (Model B) → AIC prefers the more negative value → **Model A wins on AIC**. BIC = -3224.6 (Model B) is more negative than -3218.2 (Model A) → **Model B wins on BIC**. BIC penalises extra parameters more heavily than AIC, so it favoured the simpler Model B. When they disagree, use AIC for prediction, BIC for model parsimony.

③ Common Mistake

Thinking “lower is always better” without checking the sign. Both AIC and BIC: the **more negative** value is better (lower is better in absolute terms). Here $-3241.5 < -3238.7$ (more negative), so Model A wins on AIC. Comparing absolute magnitudes without signs leads to wrong conclusions.

④ Real World Link

This AIC vs BIC disagreement is extremely common in practice. For MRF forecasting where you care about prediction accuracy, choose by AIC. For academic reporting where model parsimony matters, BIC is the more defensible choice. Always report both.

M2.022 Find the Error MEDIUM

Topic: Forecasting with ARIMA — wrong forecast horizon type

This code runs but produces only 1 forecast instead of 30. Find the error.

```

1 from statsmodels.tsa.arima.model import ARIMA
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
4                 end="2024-12-31")
5 returns = np.log(mrf["Close"]).diff().dropna()
6
7 model = ARIMA(returns, order=(1, 0, 1))
8 fit = model.fit()
9
10 # Forecast 30 steps ahead
11 forecast_result = fit.get_forecast(steps="30") # ERROR
12 forecast_mean = forecast_result.predicted_mean
13
14 print(f"Forecast length: {len(forecast_mean)}") # prints
15     1, not 30

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import warnings
6 warnings.filterwarnings('ignore')
7 from statsmodels.tsa.arima.model import ARIMA
8
9 mrf = yf.download("MRF.NS", start="2019-01-01",
10                 end="2024-12-31")
11 returns = np.log(mrf["Close"]).diff().dropna()
12
13 model = ARIMA(returns, order=(1, 0, 1))
14 fit = model.fit()
15
16 # FIX: steps must be an INTEGER, not a string

```

```

16 forecast_result = fit.get_forecast(steps=30)           # 30, not
   "30"
17 forecast_mean    = forecast_result.predicted_mean
18 conf_int         = forecast_result.conf_int()
19
20 print(f"Forecast length: {len(forecast_mean)}")      # now
   prints 30
21
22 plt.figure(figsize=(12, 4))
23 plt.plot(forecast_mean.values, label="30-Day Forecast",
   color="red")
24 plt.fill_between(range(len(forecast_mean)),
25                  conf_int.iloc[:, 0], conf_int.iloc[:, 1],
26                  alpha=0.3, color="red", label="95% CI")
27 plt.title("MRF -- ARIMA(1,0,1) 30-Day Return Forecast")
28 plt.legend(); plt.show()

```

② Plain English

`get_forecast(steps="30")` passes a string to a parameter expecting an integer. Python interprets "30" as a string, producing a single-step forecast. The fix is trivially simple but the silent failure (no crash, just wrong output) makes it easy to miss.

③ Common Mistake

Passing string arguments to numeric parameters. In Python, "30" and 30 are completely different types. Always check `type(steps)` if your forecast horizon looks wrong.

④ Real World Link

A 30-day ARIMA return forecast for MRF will show the point forecast quickly converging to the long-run mean (near zero for returns) as the horizon extends — this is a known limitation of ARIMA for financial returns. The confidence intervals widen rapidly, reflecting genuine forecast uncertainty.

Section C

Hard Problems — M2.023 to M2.034

M2.023 *Fill in the Blank* **HARD**

Topic: KPSS test — complementary to ADF

The KPSS test has the **opposite null hypothesis** to ADF. Fill in the blanks and correctly interpret both tests together.

```

1 from statsmodels.tsa.stattools import adfuller, -----
2
3 mrf = yf.download("MRF.NS", start="2019-01-01",
   end="2024-12-31")
4 close = mrf["Close"].dropna()

```

```

5 returns = np.log(close).diff()._____()
6
7 # ADF Test -- H0: Unit root (non-stationary)
8 adf = adfuller(returns)
9 print(f"ADF p-value: {adf[1]:.4f} -> "
10       f"{'Stationary' if adf[1] < 0.05 else
11         'Non-stationary'}")
12
13 # KPSS Test -- H0: _____ (opposite of ADF)
14 kpss_result = kpss(returns, regression="_____",
15                   nlags="auto")
16 print(f"KPSS stat : {kpss_result[0]:.4f}")
17 print(f"KPSS 5% CV : {kpss_result[3]['5%']:.4f}")
18
19 # KPSS: reject H0 if statistic _____ critical value
20 kpss_reject = kpss_result[0] > kpss_result[3]["5%"]
21 print(f"KPSS rejects stationarity: {kpss_reject}")
22
23 if adf[1] < 0.05 and not kpss_reject:
24     print("BOTH tests agree: STATIONARY")
25 elif adf[1] >= 0.05 and kpss_reject:
26     print("BOTH tests agree: NON-STATIONARY")
27 else:
28     print("Tests DISAGREE -- investigate further
29           (structural breaks?)")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.stattools import adfuller, kpss #
7     kpss in same module
8
9 mrf = yf.download("MRF.NS", start="2019-01-01",
10                  end="2024-12-31")
11 close = mrf["Close"].dropna()
12 returns = np.log(close).diff().dropna()
13
14 # ADF Test -- H0: Unit root (non-stationary)
15 adf = adfuller(returns)
16 print(f"ADF p-value: {adf[1]:.4f} -> "
17       f"{'Stationary' if adf[1] < 0.05 else
18         'Non-stationary'}")

```

```

16
17 # KPSS Test -- H0: STATIONARY (opposite of ADF -- null IS
    stationarity)
18 kpss_result = kpss(returns, regression="c", nlags="auto")
19 # regression="c" tests stationarity around a constant
    (level stationarity)
20 # regression="ct" tests stationarity around a constant +
    trend
21 print(f"KPSS stat : {kpss_result[0]:.4f}")
22 print(f"KPSS 5% CV : {kpss_result[3]['5%']:.4f}")
23
24 # KPSS: reject H0 (stationarity) if stat EXCEEDS the
    critical value
25 kpss_reject = kpss_result[0] > kpss_result[3]["5%"] # >
    not p-value comparison
26 print(f"KPSS rejects stationarity: {kpss_reject}")
27
28 if adf[1] < 0.05 and not kpss_reject:
29     print("BOTH tests agree: STATIONARY")
30 elif adf[1] >= 0.05 and kpss_reject:
31     print("BOTH tests agree: NON-STATIONARY")
32 else:
33     print("Tests DISAGREE -- investigate further
        (structural breaks?)")

```

② Plain English

ADF and KPSS test the same question from opposite angles. ADF's null is "series is non-stationary" — reject it ($p < 0.05$) to claim stationarity. KPSS's null is "series is stationary" — reject it ($\text{stat} > \text{CV}$) to claim non-stationarity. When both agree, you have strong evidence. When they disagree, something unusual is happening — often a structural break.

③ Common Mistake

Applying the ADF rejection rule to KPSS — using $p\text{-value} < 0.05$ for KPSS. The KPSS function returns a test statistic and critical values (not a simple $p\text{-value}$ for comparison), and you reject when the **statistic exceeds** the critical value — the opposite direction of ADF. Getting this backwards leads to completely inverted conclusions.

④ Real World Link

For MRF log returns: ADF will reject the unit root ($p \ll 0.05$, stationary), and KPSS will fail to reject stationarity ($\text{stat} < \text{CV}$). Both agree → strong confirmation that returns are stationary. This dual-test approach is the gold standard in academic time series work.

M2.024 Find the Error **HARD****Topic:** AR(p) stationarity — explosive coefficient

This code fits an AR model and produces astronomically large forecast values. Identify the root cause and the fix.

```

1  import numpy as np
2  from statsmodels.tsa.arima.model import ARIMA
3
4  # Simulated series: AR(1) with phi=1.3 (explosive)
5  np.random.seed(42)
6  phi = 1.3    # ERROR: |phi| > 1 means non-stationary,
               # explosive process
7  n    = 300
8  y    = np.zeros(n)
9  y[0] = 1.0
10 for t in range(1, n):
11     y[t] = phi * y[t-1] + np.random.randn()
12
13 # Fit AR(1)
14 model = ARIMA(y, order=(1, 0, 0))
15 fit    = model.fit()
16 print(f"Estimated phi: {fit.params['ar.L1']:.4f}")
17
18 # Forecast 20 steps
19 forecast = fit.forecast(steps=20)
20 print(f"Forecast step 1: {forecast.iloc[0]:.2f}")
21 print(f"Forecast step 20: {forecast.iloc[-1]:.2f}")    #
               will be enormous

```

Explain: (i) why this happens, (ii) what the correct fix is for a real dataset.

Solution**① Correct Code & Explanation**

```

1  !pip install yfinance statsmodels arch -q
2  import numpy as np
3  import warnings
4  warnings.filterwarnings('ignore')
5  from statsmodels.tsa.arima.model import ARIMA
6  from statsmodels.tsa.stattools import adfuller
7
8  # ROOT CAUSE:
9  # phi=1.3 means |phi| > 1 -> AR(1) process is EXPLOSIVE
   # (non-stationary)
10 # Each step multiplies previous value by 1.3 -> series
   # grows to infinity
11 # Forecasts from an explosive process diverge geometrically
12
13 np.random.seed(42)

```

```

14 phi = 1.3
15 n = 300
16 y = np.zeros(n)
17 y[0] = 1.0
18 for t in range(1, n):
19     y[t] = phi * y[t-1] + np.random.randn()
20
21 # ADF CONFIRMS the problem
22 adf = adfuller(y)
23 print(f"ADF p-value: {adf[1]:.4f}") # will be >> 0.05 --
    non-stationary!
24
25 # FIX FOR REAL DATA:
26 # Step 1: Always run ADF BEFORE fitting any AR model
27 # Step 2: If p > 0.05, DIFFERENCE the series until
    stationary
28 y_diff = np.diff(y) # first difference
29
30 adf_diff = adfuller(y_diff)
31 print(f"ADF on differenced series: {adf_diff[1]:.6f}") #
    now stationary
32
33 # Step 3: Fit ARIMA with d=1 instead of AR(1) with d=0
34 model_fixed = ARIMA(y, order=(1, 1, 0)) # d=1 handles
    non-stationarity
35 fit_fixed = model_fixed.fit()
36 forecast_fixed = fit_fixed.forecast(steps=20)
37 print(f"\nFixed forecast step 20:
    {forecast_fixed.iloc[-1]:.4f}") # reasonable

```

② Plain English

The fundamental error is fitting a stationary model (AR(1) with $d = 0$) to a non-stationary explosive series. When the estimated $\varphi > 1$, each forecast multiplies the last value by more than 1, growing exponentially to infinity. The fix is to always confirm stationarity with ADF first, then difference if needed (use $d = 1$).

③ Common Mistake

Trusting model output without sanity-checking forecasts. If your 20-day forecast for MRF returns jumps from 0.001 to 5,000,000, that's a sign something went catastrophically wrong in model specification — not a sign MRF is about to moon. Always plot forecasts and check they look plausible given historical values.

④ Real World Link

Explosive AR processes don't occur in real financial returns (markets correct them), but they can appear in levels if you forget to difference. If you accidentally fit AR(1) to MRF prices (not returns) and get $\varphi \approx 0.998$, your 6-month forecast might look reasonable — but it's built on a technically non-stationary foundation.

M2.025 Multiple Choice **HARD****Topic:** ARMA invertibility condition

An MA(1) process is $X_t = \varepsilon_t + \theta \cdot \varepsilon_{t-1}$. For the process to be **invertible**, which condition must hold?

- (A) $|\theta| < 1$ — the absolute value of the MA coefficient must be less than 1
- (B) $\theta > 0$ — only positive MA coefficients are invertible
- (C) $|\theta| > 1$ — the MA coefficient must exceed 1 in magnitude
- (D) $\theta = 0$ — the process degenerates to white noise

Solution**① Correct Answer: (A)****② Plain English**

Invertibility for MA(q) mirrors stationarity for AR(p) — both require the roots of their respective polynomials to lie outside the unit circle, which for MA(1) translates to $|\theta| < 1$. An invertible MA process can be written as an infinite AR process with geometrically decaying coefficients. Non-invertible MA processes ($|\theta| > 1$) have a statistically equivalent invertible representation (replace θ with $1/\theta$) — statsmodels enforces invertibility automatically.

③ Common Mistake

Confusing stationarity (AR condition) with invertibility (MA condition). They are parallel concepts: AR(p) requires $|\varphi| < 1$ for AR(1). MA(q) requires $|\theta| < 1$ for MA(1). Remember: stationarity $\leftrightarrow$ AR, invertibility $\leftrightarrow$ MA.

④ Real World Link

When you fit ARIMA with `enforce_invertibility=True` (the default in statsmodels SARIMAX), the optimizer constrains θ to $|\theta| < 1$ during estimation. If you set `enforce_invertibility=False` for a SARIMA model, you're allowing potentially non-invertible solutions — sometimes necessary for convergence but should be checked in the summary output.

M2.026 Write from Scratch **HARD****Topic:** SARIMA grid search with AIC selection

Write complete code that: (1) downloads MRF monthly prices from 2019, (2) performs a grid search over SARIMA(p,1,q)(P,1,Q)[12] for $p, q \in \{0, 1, 2\}$ and $P, Q \in \{0, 1\}$, (3) stores AIC for each combination, (4) prints a sorted leaderboard of top 5 models by AIC, (5) fits the best model and prints a summary.

Solution**① Correct Code**

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
```

```

3 import numpy as np
4 import pandas as pd
5 import itertools
6 import warnings
7 warnings.filterwarnings('ignore')
8 from statsmodels.tsa.statespace.sarimax import SARIMAX
9
10 # Step 1: Download MRF monthly data
11 mrf = yf.download("MRF.NS", start="2019-01-01",
12                  end="2024-12-31")
13 monthly = mrf["Close"].dropna().resample("ME").last()
14
15 # Step 2: Define grid search space
16 p_values = [0, 1, 2]; q_values = [0, 1, 2]
17 P_values = [0, 1]; Q_values = [0, 1]
18 d = 1; D = 1; s = 12
19
20 grid = list(itertools.product(p_values, q_values,
21                               P_values, Q_values))
22 print(f"Total models to evaluate: {len(grid)}")
23
24 # Step 3: Grid search
25 results = []
26 for (p, q, P, Q) in grid:
27     try:
28         model = SARIMAX(monthly,
29                         order=(p, d, q),
30                         seasonal_order=(P, D, Q, s),
31                         enforce_stationarity=False,
32                         enforce_invertibility=False)
33         fit = model.fit(dispatch=False)
34         results.append({
35             "order":
36                 f"SARIMA({p},{d},{q})({P},{D},{Q})[{s}]",
37             "p": p, "q": q, "P": P, "Q": Q,
38             "AIC": fit.aic, "BIC": fit.bic
39         })
40     except Exception:
41         results.append({
42             "order":
43                 f"SARIMA({p},{d},{q})({P},{D},{Q})[{s}]",
44             "AIC": np.inf, "BIC": np.inf
45         })
46
47 # Step 4: Leaderboard
48 df_results =
49     pd.DataFrame(results).sort_values("AIC").reset_index(drop=True)
50 print("\n" + "="*65)
51 print(" SARIMA Grid Search -- Top 5 Models by AIC")

```

```

47 print("="*65)
48 print(df_results[["order", "AIC",
49                  "BIC"]].head(5).to_string(index=True))
50 # Step 5: Fit best model
51 best = df_results.iloc[0]
52 best_p, best_q = int(best["p"]), int(best["q"])
53 best_P, best_Q = int(best["P"]), int(best["Q"])
54
55 print(f"\nBest model: {best['order']}")
56 best_fit = SARIMAX(monthly,
57                    order=(best_p, d, best_q),
58                    seasonal_order=(best_P, D, best_Q, s),
59                    enforce_stationarity=False,
60                    enforce_invertibility=False).fit(dispatch=False)
61 print(best_fit.summary())

```

② Plain English

Grid search systematically tries every combination of (p,q,P,Q) and ranks them by AIC. This is the computational equivalent of the “read ACF/PACF and guess” approach — you let the algorithm check all options and pick the winner. It’s the basis of how `auto_arima()` works internally.

③ Common Mistake

Not wrapping the model fitting in `try/except` — some (p,q,P,Q) combinations fail to converge (singular matrices, optimization failures). Without error handling, one failed combination crashes the entire grid search. The `try/except` assigns `AIC=inf` (worst score), ensuring the loop completes.

④ Real World Link

Professional quant libraries (pmdarima’s `auto_arima`, sktime’s `AutoARIMA`) implement exactly this logic with additional heuristics to skip clearly bad combinations. Understanding the manual version means you know what these automated tools are doing — and you can debug them when they give unexpected results.

M2.027 Predict the Output **HARD**

Topic: AR(1) — forecast convergence to mean

Without running this code, predict the **pattern** of the 10-step forecast, and estimate the approximate final value.

```

1 import numpy as np
2 from statsmodels.tsa.arima.model import ARIMA
3
4 np.random.seed(7)
5 phi = 0.6
6 n = 500
7 y = np.zeros(n)
8 for t in range(1, n):

```

```

9     y[t] = phi * y[t-1] + np.random.randn()
10
11 fit = ARIMA(y, order=(1, 0, 0)).fit()
12
13 forecast = fit.forecast(steps=10)
14 print("Step | Forecast")
15 for i, val in enumerate(forecast, 1):
16     print(f" {i:2d} | {val:+.6f}")

```

Predict: (i) will the forecast values increase, decrease, or do something more specific? (ii) what will the forecast be approaching at step 10?

Solution

① Predicted Output Pattern

```

Step | Forecast
1 | [close to  $\varphi \times y_{-1}$ , approximately  $\pm 0.3$  to  $\pm 0.8$ ]
2 | [smaller magnitude than step 1]
3 | [smaller still...]
...
10 | [very close to 0.000000]

```

Pattern: Geometric decay toward zero (the process mean).

```

1 # AR(1) forecast formula (h steps ahead from time T):
2 #  $\hat{y}_{T+h} = \phi^h * y_T$  (for zero-mean AR(1))
3 #
4 #  $\phi = 0.6$ , so:
5 # Step 1:  $0.6^1 * y_T = 0.600 * y_T$ 
6 # Step 2:  $0.6^2 * y_T = 0.360 * y_T$ 
7 # Step 3:  $0.6^3 * y_T = 0.216 * y_T$ 
8 # ...
9 # Step 10:  $0.6^{10} * y_T = 0.006 * y_T \approx 0$ 
10 #
11 print(f"0.6^10 = {0.6**10:.6f}") # 0.006047 -- very close
    to zero
12 # The 10-step forecast is 99.4% decayed toward the
    unconditional mean (0)

```

② Plain English

For a zero-mean AR(1) with $|\varphi| < 1$, the h -step forecast formula is $\varphi^h \times y_T$. As h increases, φ^h decays geometrically to zero — so all forecasts converge to the unconditional mean (zero here). The rate of convergence depends on φ : smaller $\varphi \rightarrow$ faster convergence; φ closer to 1 $\rightarrow$ slower convergence (persistence).

③ Common Mistake

Expecting the forecast to stay flat at the last observed value, or to continue trending in the direction of the last move. AR(1) forecasts always converge to the mean — that's the defining property of a stationary mean-reverting process. Only non-stationary (unit root) processes have forecasts that don't revert.

④ Real World Link

This mean-reversion property is why ARIMA return forecasts for MRF quickly converge to ≈ 0 (the approximate long-run daily return). Beyond a few lags, the model is basically saying "I don't know" — which is honest, and why forecast confidence intervals widen with horizon.

M2.028 *Fill in the Blank* **HARD**

Topic: ARIMA on MRF — full pipeline with forecast plot

Fill in all blanks to complete a full ARIMA pipeline with confidence intervals.

```

1 from statsmodels.tsa.arima.model import ARIMA
2 from statsmodels.tsa.stattools import adfuller
3
4 mrf = yf.download("MRF.NS", start="2019-01-01",
5                   end="2024-12-31")
6 close = mrf["_____"].dropna()
7 returns = np.log(close)._____.dropna()
8
9 # 1. Confirm stationarity
10 adf_p = adfuller(returns)[_____] # get p-value
11 assert adf_p < 0.05, f>Returns not stationary!
12     p={adf_p:.4f}"
13
14 # 2. Fit ARIMA(1,0,1)
15 fit = ARIMA(returns, order=(1, _____, 1)).fit()
16
17 # 3. Forecast 20 steps with confidence intervals
18 fc_obj = fit._____(steps=20)
19 fc_mean = fc_obj._____
20 fc_ci = fc_obj.conf_int(alpha=0.05)
21
22 # 4. Plot
23 fig, ax = plt.subplots(figsize=(14, 5))
24 ax.plot(returns[-_____:], label="Observed Returns",
25         color="steelblue")
26
27 x_fc = range(len(returns), len(returns) + _____)
28 ax.plot(x_fc, fc_mean, color="red", label="Forecast",
29         linewidth=2)
30 ax.fill_between(x_fc, fc_ci.iloc[:, 0], fc_ci.iloc[:,
31                 _____],

```

```

27         color="red", alpha=0.2, label="95% CI")
28 ax.legend()
29 ax.set_title("MRF Log Returns -- ARIMA(1,0,1) Forecast")
30 plt.show()

```

Solution

① Correct Code

```

1  !pip install yfinance statsmodels arch -q
2  import yfinance as yf
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import warnings
6  warnings.filterwarnings('ignore')
7  from statsmodels.tsa.arima.model import ARIMA
8  from statsmodels.tsa.stattools import adfuller
9
10 mrf = yf.download("MRF.NS", start="2019-01-01",
11                  end="2024-12-31")
12 close = mrf["Close"].dropna()           # "Close" column
13 returns = np.log(close).diff().dropna() # .diff() for
14                                           first difference
15
16 # 1. Confirm stationarity
17 adf_p = adfuller(returns)[1]             # index [1] =
18                                           p-value
19 assert adf_p < 0.05, f>Returns not stationary!
20 p={adf_p:.4f}"
21
22 # 2. Fit ARIMA(1,0,1)
23 fit = ARIMA(returns, order=(1, 0, 1)).fit() # d=0:
24                                           returns already stationary
25
26 # 3. Forecast 20 steps with confidence intervals
27 fc_obj = fit.get_forecast(steps=20)       #
28                                           get_forecast gives richer object
29 fc_mean = fc_obj.predicted_mean           # point
30                                           forecast Series
31 fc_ci = fc_obj.conf_int(alpha=0.05)       # DataFrame:
32                                           lower and upper CI
33
34 # 4. Plot
35 fig, ax = plt.subplots(figsize=(14, 5))
36 ax.plot(returns[-100:].values, label="Observed Returns",
37         color="steelblue")
38
39 x_fc = range(100, 100 + 20)

```

```

31 ax.plot(x_fc, fc_mean.values, color="red",
        label="Forecast", linewidth=2)
32 ax.fill_between(x_fc, fc_ci.iloc[:, 0], fc_ci.iloc[:, 1],
33                color="red", alpha=0.2, label="95% CI")
34 ax.axhline(0, color="black", linewidth=0.5, linestyle="--")
35 ax.legend()
36 ax.set_title("MRF Log Returns -- ARIMA(1,0,1) Forecast")
37 plt.tight_layout()
38 plt.show()

```

② Plain English

`get_forecast()` is more powerful than `.forecast()` — it returns a result object you can query for both point forecasts (`predicted_mean`) and confidence intervals (`conf_int()`). The confidence intervals widen with each step because forecast uncertainty accumulates over time.

③ Common Mistake

Using `.forecast(steps=20)` instead of `.get_forecast(steps=20)` when you need confidence intervals. `.forecast()` only returns point forecasts. `.get_forecast()` returns a rich object — always use `get_forecast` when plotting forecast uncertainty.

④ Real World Link

The widening confidence intervals in the MRF forecast plot are not a weakness — they're the model being honest about what it doesn't know. A model that shows narrow confidence intervals beyond 10 days is probably miscalibrated. Wide intervals at long horizons are statistically correct for ARIMA return models.

M2.029 Find the Error **HARD**

Topic: Seasonal differencing — wrong resample + model mismatch

This code has **three errors** — the data preparation, the seasonal period, and the stationarity check are all wrong. Find all three.

```

1 from statsmodels.tsa.statespace.sarimax import SARIMAX
2 from statsmodels.tsa.stattools import adfuller
3
4 mrf = yf.download("MRF.NS", start="2019-01-01",
5                  end="2024-12-31")
6 close = mrf["Close"].dropna()
7
8 # ERROR 1: resample to weekly but fit monthly model below
9 weekly = close.resample("W").last()
10
11 # ERROR 2: fitting s=12 on WEEKLY data (wrong period)
12 model = SARIMAX(close,
13                 order=(1, 1, 1),
14                 seasonal_order=(1, 1, 1, 12)) # s=12
15                 # wrong for weekly data
16 fit = model.fit(dis= False)

```

```

15
16 # ERROR 3: ADF on raw prices after fitting -- should be
    BEFORE, on stationary series
17 adf = adfuller(close)      # checking raw prices post-hoc
    -- wrong target and timing
18 print(f"ADF p-value: {adf[1]:.4f}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.statespace.sarimax import SARIMAX
7 from statsmodels.tsa.stattools import adfuller
8
9 mrf = yf.download("MRF.NS", start="2019-01-01",
    end="2024-12-31")
10 close = mrf["Close"].dropna()
11
12 # FIX 1: Resample to MONTHLY to match s=12 seasonal period
13 monthly = close.resample("ME").last() # monthly, not
    weekly
14
15 # FIX 3 (do FIRST before fitting): ADF on log-prices to
    check differencing needed
16 log_monthly = np.log(monthly)
17 adf_before = adfuller(log_monthly)
18 print(f"ADF on log monthly prices: p =
    {adf_before[1]:.4f}") # expect > 0.05
19
20 log_returns_monthly = log_monthly.diff().dropna()
21 adf_after = adfuller(log_returns_monthly)
22 print(f"ADF on log returns: p = {adf_after[1]:.4f}") #
    expect < 0.05 -> d=1
23
24 # FIX 2: Use s=12 correctly -- NOW matches monthly data
    with annual season
25 model = SARIMAX(monthly,
26                  order=(1, 1, 1),
27                  seasonal_order=(1, 1, 1, 12), # s=12
    correct for monthly + annual
28                  enforce_stationarity=False,
29                  enforce_invertibility=False)
30 fit = model.fit(dispatch=False)

```



```
31 print(fit.summary())
```

② Plain English

Three sequential errors: (1) resampling to weekly but using $s = 12$ (annual in monthly) — the period must match the data frequency; (2) $s = 12$ on weekly data means 12-week cycles, not 12-month cycles; (3) running ADF after fitting is backwards — stationarity testing guides model specification and must happen first. Always: test $\rightarrow$ specify $\rightarrow$ fit $\rightarrow$ validate.

③ Common Mistake

Mixing data frequencies across the pipeline. If you resample to weekly, all downstream analysis must use weekly conventions: $s = 52$ for annual, $s = 5$ for daily-within-a-week. Switching resample frequency without updating s is a silent error — the model fits without crashing but produces meaningless seasonal estimates.

④ Real World Link

In real analysis, these three errors together would give you a SARIMA model that appears to have good in-sample fit but fails completely in out-of-sample validation. The seasonal component would be capturing 12-week cycles in weekly MRF data — which has no financial meaning — rather than the genuine annual sales seasonality you intended to model.

M2.030 Short Answer **HARD**

Topic: ARIMA vs SARIMA — when to choose

In 2 sentences, explain when you would choose SARIMA over ARIMA for MRF data, and identify what evidence in the ACF plot would confirm that a seasonal model is needed.

Solution

① Model Answer

Choose SARIMA over ARIMA when the data exhibits repeating patterns at regular seasonal intervals — for MRF monthly data, this means the ACF of the series shows significant spikes at seasonal lags (lag 12, lag 24, etc.) that plain ARIMA's non-seasonal terms cannot capture. The key evidence in the ACF plot is significant spikes specifically at multiples of the seasonal period s (e.g., lags 12, 24 for monthly data) alongside the usual low-lag structure — if those seasonal spikes are absent, plain ARIMA is sufficient.

② Common Mistake

Adding seasonal terms “just in case” without checking for seasonal ACF spikes. SARIMA adds parameters — if there's no genuine seasonal structure in the data, you're introducing noise and inflating AIC. Always let the ACF/PACF at seasonal lags guide the decision. Significant spike at lag 12 $\rightarrow$ add seasonal component. Clean at lag 12 $\rightarrow$ stick with ARIMA.

③ Real World Link

For MRF on monthly data, genuine seasonal spikes in the ACF reflect the annual tyre sales cycle (summer peak). On daily data, you'd look for spikes at lag 5 (weekly) or lag 252 (annual). If those spikes aren't there, adding `seasonal_order=(1,1,1,12)` won't improve forecasts and may actually hurt by overfitting.

M2.031 Write from Scratch **HARD**

Topic: ARIMA residual full diagnostic suite

Write a reusable function `arima_diagnostics(fit, title)` that, given a fitted ARIMA model: (1) plots 4 residual diagnostics in a 2×2 grid: residual time series, residual ACF, residual histogram with normal overlay, Q-Q plot; (2) runs the Ljung-Box test (20 lags) and prints structured results; (3) prints a model verdict with suggested next steps.

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import scipy.stats as stats
6 import warnings
7 warnings.filterwarnings('ignore')
8 from statsmodels.tsa.arima.model import ARIMA
9 from statsmodels.graphics.tsaplots import plot_acf
10 from statsmodels.stats.diagnostic import acorr_ljungbox
11
12 def arima_diagnostics(fit, title="ARIMA Model"):
13     """Full residual diagnostic suite for a fitted
14         statsmodels ARIMA model."""
15     resid = fit.resid.dropna()
16
17     # 2x2 Diagnostic Plot
18     fig, axes = plt.subplots(2, 2, figsize=(14, 8))
19     fig.suptitle(f"Residual Diagnostics -- {title}",
20                 fontsize=13, fontweight="bold")
21
22     # (a) Residual time series
23     ax = axes[0, 0]
24     ax.plot(resid, color="steelblue", linewidth=0.7,
25             alpha=0.8)
26     ax.axhline(0, color="red", linewidth=1, linestyle="--")
27     ax.set_title("Residual Time Series")
28     ax.set_ylabel("Residual")

```

```

26
27 # (b) ACF of residuals -- want all inside bands
28 ax = axes[0, 1]
29 plot_acf(resid, lags=30, alpha=0.05, ax=ax, zero=False)
30 ax.set_title("ACF of Residuals (want: all inside
    bands)")
31
32 # (c) Histogram with normal overlay
33 ax = axes[1, 0]
34 ax.hist(resid, bins=60, density=True,
    color="steelblue", alpha=0.5, label="Residuals")
35 x = np.linspace(resid.min(), resid.max(), 300)
36 ax.plot(x, stats.norm.pdf(x, resid.mean(),
    resid.std()),
    color="red", linewidth=2, label="Normal Fit")
37 ax.set_title("Residual Distribution vs Normal")
38 ax.legend(fontsize=9)
39
40
41 # (d) Q-Q plot -- points should lie on the diagonal
42 ax = axes[1, 1]
43 stats.probplot(resid, dist="norm", plot=ax)
44 ax.set_title("Q-Q Plot (want: points on diagonal)")
45
46 plt.tight_layout()
47 plt.show()
48
49 # Ljung-Box Test
50 lb = acorr_ljungbox(resid, lags=20, return_df=True)
51 n_significant = (lb["lb_pvalue"] < 0.05).sum()
52 min_pval = lb["lb_pvalue"].min()
53
54 print(f"\n{'='*55}")
55 print(f"  Ljung-Box Residual Test -- {title}")
56 print(f"{'='*55}")
57 print(f"  Significant lags : {n_significant}/20  (p <
    0.05)")
58 print(f"  Minimum p-value   : {min_pval:.4f}")
59 print(f"  AIC                  : {fit.aic:.2f}")
60 print(f"  BIC                  : {fit.bic:.2f}")
61
62 # Verdict
63 print(f"\n{'-'*55}")
64 if n_significant == 0 and min_pval > 0.10:
65     print("  VERDICT: Well-specified")
66     print("  Residuals consistent with white
        noise.")
67     print("  Model is ready for forecasting.")
68 elif n_significant <= 2:
69     print("  VERDICT: Marginally acceptable")

```

```

70         print("      Suggestion: Try increasing AR or MA
           order by 1.")
71     else:
72         print("  VERDICT: Misspecified -- residuals have
           structure")
73         print("  Next steps:")
74         print("      1. Increase AR order p by 1")
75         print("      2. Increase MA order q by 1")
76         print("      3. Check for ARCH effects (GARCH in
           Module 4)")
77         print("      4. Consider seasonal model (SARIMA)")
78     print(f"{'='*55}\n")
79
80 # USAGE
81 mrf      = yf.download("MRF.NS", start="2019-01-01",
           end="2024-12-31")
82 returns = np.log(mrf["Close"]).diff().dropna()
83 fit = ARIMA(returns, order=(1, 0, 1)).fit()
84 arima_diagnostics(fit, title="ARIMA(1,0,1) -- MRF Log
           Returns")

```

② Plain English

This function packages every post-estimation check into a single call. The 2×2 plot gives visual evidence (ACF, histogram, Q-Q), and Ljung-Box gives statistical evidence. The verdict logic translates numbers into actionable guidance — which is what separates a junior analyst who prints numbers from a senior analyst who interprets them.

③ Common Mistake

Only looking at one diagnostic. A model can pass the Ljung-Box test but have heavy-tailed residuals (Q-Q plot shows curved tails) — suggesting GARCH effects even without mean autocorrelation. A complete diagnostic requires all four plots. Use this function for every model you fit, without exception.

④ Real World Link

In a professional research note on MRF, you would attach this diagnostic grid to prove your model is correctly specified. Reviewers and regulators in financial modelling explicitly require residual analysis — a model without it is an incomplete model, regardless of how impressive the AIC looks.

M2.032 Multiple Choice **HARD**

Topic: Seasonal ARIMA — notation interpretation

A colleague hands you the notation SARIMA(2,1,1)(1,1,0)[12] fitted to monthly MRF prices. What does this model contain in total?

- (A) 2 AR lags, 1 differencing, 1 MA lag — seasonal components are at lag 12
- (B) 3 AR parameters, 2 differencing operations, 1 MA parameter, with all seasonal parameters at multiples of lag 12

- (C) 2 non-seasonal AR, once non-seasonal differenced, 1 non-seasonal MA, 1 seasonal AR, once seasonally-differenced, 0 seasonal MA — all seasonal effects at lag multiples of 12
- (D) The model is equivalent to ARIMA(2,1,1) applied independently for each of the 12 months

Solution

① **Correct Answer:** (C)

② **Plain English**

SARIMA(p,d,q)(P,D,Q)[s] decomposes cleanly: non-seasonal part (2,1,1) means 2 AR lags at lags 1, 2; differenced once; 1 MA term at lag 1. Seasonal part (1,1,0)[12] means 1 seasonal AR term at lag 12; seasonally differenced once (subtract value from 12 months ago); 0 seasonal MA terms. The total differencing is $d + D = 1 + 1 = 2$ — so the model differences twice in total (once at lag 1, once at lag 12).

③ **Common Mistake**

Choosing (D) — SARIMA is emphatically NOT 12 separate ARIMA models. It's one unified model that captures both short-run dynamics (non-seasonal terms) and annual cycles (seasonal terms) simultaneously. The seasonal AR and MA terms appear at lag multiples of s (12, 24, 36...), not as separate sub-models.

④ **Real World Link**

For MRF monthly data, SARIMA(2,1,1)(1,1,0)[12] is a reasonable candidate. The two non-seasonal AR terms capture autocorrelation in consecutive months. The seasonal AR term at lag 12 captures the fact that this month's price is correlated with the same month last year — a genuine pattern in tyre demand driven by recurring seasonal factors.

M2.033 Predict the Output **HARD**

Topic: ARIMA vs random walk — AIC comparison

Without running this code, predict which model will have lower AIC, and explain why.

```

1 import numpy as np
2 from statsmodels.tsa.arima.model import ARIMA
3
4 np.random.seed(99)
5 phi = 0.7
6 n = 400
7 y = np.zeros(n)
8 for t in range(1, n):
9     y[t] = phi * y[t-1] + np.random.randn()
10
11 # Model 1: Correct specification -- AR(1)
12 fit_ar1 = ARIMA(y, order=(1, 0, 0)).fit()
13
```

```

14 # Model 2: Null model -- no AR terms
15 fit_rw = ARIMA(y, order=(0, 0, 0)).fit()
16
17 print(f"AR(1) AIC: {fit_ar1.aic:.2f}")
18 print(f"Null AIC: {fit_rw.aic:.2f}")
19 print(f"AR(1) wins: {fit_ar1.aic < fit_rw.aic}")

```

Predict: which model wins on AIC, and by approximately how much?

Solution

① Predicted Output

```

AR(1) AIC: [lower value, approximately 50--150 points less than
Null]
Null AIC: [higher value]
AR(1) wins: True

```

```

1 # Reasoning:
2 # With phi=0.7 (strong AR coefficient) and n=400
  observations:
3 #
4 # AR(1) captures genuine structure -> residuals have
  variance ~1.0
5 # Null model ignores the 0.7 autocorrelation -> residuals
  still contain
6 # the AR structure -> much higher variance -> much lower
  log-likelihood
7 # -> much higher AIC
8 #
9 # AIC penalises 1 extra parameter (the AR coefficient), but
10 # rewards the much better fit -> net: AR(1) wins
  substantially
11 #
12 # Approx AIC difference: n * log(sigma^2_AR1 /
  sigma^2_Null) - 2
13 # = 400 * log(1.0 / variance_of_null_residuals) -- large
  positive number

```

② Plain English

When the true process is AR(1) with $\varphi = 0.7$, the data has strong serial correlation that the null model (white noise) completely ignores. The AR(1) model fits dramatically better — its residuals have much lower variance — and the AIC improvement easily outweighs the 2-unit penalty for one extra parameter.

③ Common Mistake

Thinking AIC always penalises complex models enough to prefer simpler ones. AIC's penalty ($2k$ for k parameters) is calibrated to prevent *marginal* improvements from justifying extra parameters. But when the AR coefficient is genuinely 0.7 and you have 400 observations, the evidence for the AR term is overwhelming — the penalty is trivially small compared to the fit improvement.

④ Real World Link

For MRF returns (where the true AR coefficient is close to 0, not 0.7), the AIC comparison between AR(1) and white noise will be much closer. The AR(1) might win by only 1–2 AIC units — barely meaningful. This reflects reality: MRF returns are close to a random walk, and simple models often win on AIC because there's little genuine AR structure to capture.

M2.034 Fill in the Blank **HARD**

Topic: Transfer function — exogenous variable in SARIMAX

Fill in the blanks to fit a SARIMAX model using BTC-USD as a (simplified) exogenous variable, demonstrating the SARIMAX exog syntax.

```

1 from statsmodels.tsa.statespace.sarimax import SARIMAX
2
3 # Download both series
4 mrf = yf.download("MRF.NS", start="2020-01-01",
5                   end="2024-12-31")
6 btc = yf.download("_____", start="2020-01-01",
7                   end="2024-12-31")
8
9 # Compute log returns
10 mrf_ret = np.log(mrf["Close"]).diff()
11 btc_ret = np.log(btc["_____"]).diff()
12
13 # Inner join -- keep only dates where both have data
14 combined =
15     mrf_ret.to_frame("MRF").join(btc_ret.to_frame("BTC"),
16                                 how="_____").dropna()
17
18 mrf_y = combined["MRF"]      # endogenous (dependent)
19                               variable
20 btc_x = combined["_____"] # exogenous (independent)
21                               variable
22
23 # Fit SARIMAX with exogenous variable
24 model = SARIMAX(mrf_y,
25                 exog=_____,      # pass the exogenous
26                       series here
27                 order=(1, 0, 1),

```

```

21         trend="c")
22 fit = model.fit(_____)
23
24 print(fit.summary())
25 print(f"\nBTC coefficient: {fit.params['_____']:.6f}")

```

Solution

① Correct Code

```

1  !pip install yfinance statsmodels arch -q
2  import yfinance as yf
3  import numpy as np
4  import warnings
5  warnings.filterwarnings('ignore')
6  from statsmodels.tsa.statespace.sarimax import SARIMAX
7
8  # Download both series
9  mrf = yf.download("MRF.NS", start="2020-01-01",
10                  end="2024-12-31")
11  btc = yf.download("BTC-USD", start="2020-01-01",
12                  end="2024-12-31") # exogenous proxy
13
14 # Compute log returns
15 mrf_ret = np.log(mrf["Close"]).diff()
16 btc_ret = np.log(btc["Close"]).diff()
17
18 # Inner join -- keep only common trading dates
19 combined =
20     mrf_ret.to_frame("MRF").join(btc_ret.to_frame("BTC"),
21     how="inner").dropna()
22
23 mrf_y = combined["MRF"] # what we're modelling
24     (endogenous)
25 btc_x = combined["BTC"] # external predictor (exogenous)
26
27 # Fit SARIMAX with BTC as exogenous variable
28 model = SARIMAX(mrf_y,
29                 exog=btc_x, # exog = exogenous
30                     variable(s), same length as endog
31                     order=(1, 0, 1),
32                     trend="c") # "c" = include
33                     intercept/constant
34 fit = model.fit(dispatch=False)
35
36 print(fit.summary())
37 print(f"\nBTC coefficient: {fit.params['BTC']:.6f}")

```



```
31 # Interpretation: if positive, MRF and BTC move together;
    if ~0, little relationship
```

② Plain English

SARIMAX extends SARIMA by allowing external (exogenous) variables — other time series that help explain the target. The `exog` parameter receives a series or DataFrame of predictors aligned to the same dates as your target. The fitted coefficient tells you how much a 1-unit change in the exogenous variable shifts the target, controlling for its own ARMA dynamics.

③ Common Mistake

Passing `exog` that's not aligned to the same dates as the endogenous variable. If MRF has 1250 rows and BTC has 1260 (BTC trades weekends), the mismatch causes a dimension error. The inner join solves this — keep only dates where both series have observations.

④ Real World Link

In a real MRF model, rubber futures prices or crude oil prices would be the genuine exogenous variables. BTC is used here purely to demonstrate the syntax. The methodology is identical — what matters is whether the exogenous variable has genuine predictive power for MRF returns after controlling for own-lag dynamics.

Section D ★ Exam Style Problems — M2.035 to M2.040

M2.035 Exam Style — Unit Root Testing and Differencing Strategy EXAM

Marks: 15 | Time: 25 minutes | Dataset: MRF.NS

Part A — ADF Test on Price Level [4 marks]

Download MRF data (2019–2024). Run the ADF test on the raw closing price. Print the test statistic, p-value, and critical values at 1%, 5%, and 10%. State your conclusion about stationarity with explicit reference to the p-value.

Part B — Log Transformation and Re-test [4 marks]

Apply a log transformation to MRF closing prices. Run ADF again. Compare the p-value with Part A. Explain in a code comment whether the log transformation alone achieves stationarity.

Part C — First Differencing and Final Confirmation [4 marks]

Apply first differencing to the log price series. Run ADF a third time. State the final value of `d` to use in ARIMA. Plot the three series — raw price, log price, log returns — in a 3×1 subplot.

Part D — KPSS Confirmation [3 marks]

Run the KPSS test on MRF log returns. Report whether KPSS and ADF agree. Write a 2-line comment explaining the null hypotheses of each test.

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import warnings
6 warnings.filterwarnings('ignore')
7 from statsmodels.tsa.stattools import adfuller, kpss
8
9 mrf = yf.download("MRF.NS", start="2019-01-01",
10                  end="2024-12-31")
11 close = mrf["Close"].dropna()
12
13 # PART A -- ADF on Raw Price Level [4 marks]
14
15 adf_raw = adfuller(close)
16 print("PART A -- ADF Test: MRF Raw Closing Price")
17 print(f" Test Statistic : {adf_raw[0]:.4f}")
18 print(f" p-value : {adf_raw[1]:.4f}")
19 print(f" Critical 1% : {adf_raw[4]['1%']:.4f}")
20 print(f" Critical 5% : {adf_raw[4]['5%']:.4f}")
21 print(f" Critical 10% : {adf_raw[4]['10%']:.4f}")
22 # p-value >> 0.05 -> FAIL to reject H0 (unit root) ->
23 # NON-STATIONARY
24 if adf_raw[1] > 0.05:
25     print(" Conclusion: NON-STATIONARY (p > 0.05, unit
26           root not rejected)")
27
28 # PART B -- Log Transformation [4 marks]
29
30 log_price = np.log(close)
31 adf_log = adfuller(log_price)
32 print(f"\nPART B -- ADF Test: Log(MRF Price)")
33 print(f" p-value (raw) : {adf_log[1]:.4f}")
34 print(f" p-value (log) : {adf_log[1]:.4f}")
35 # Log reduces scale but does NOT achieve stationarity
36 # because the MEAN of log prices still trends upward
37 # Log transform stabilises variance but not mean -> still
38 # non-stationary
39 if adf_log[1] > 0.05:
40     print(" -> Log alone does NOT achieve stationarity
41           (mean still trends up)")
42
43 # PART C -- First Differencing [4 marks]
44
45 log_returns = log_price.diff().dropna()
46 adf_ret = adfuller(log_returns)
47 print(f"\nPART C -- ADF Test: MRF Log Returns")

```

```

40 print(f"    p-value: {adf_ret[1]:.6f}")
41 if adf_ret[1] < 0.05:
42     print("    STATIONARY -- log returns pass ADF test")
43     print("    -> Recommended ARIMA parameter: d = 1")
44
45 fig, axes = plt.subplots(3, 1, figsize=(14, 10))
46 axes[0].plot(close,          color="steelblue",
47              linewidth=0.8)
48 axes[0].set_title("Raw MRF Price (non-stationary --
49                  trending mean)")
50 axes[0].set_ylabel("INR")
51 axes[1].plot(log_price,     color="darkorange",
52              linewidth=0.8)
53 axes[1].set_title("Log(MRF Price) (non-stationary -- still
54                  trending)")
55 axes[1].set_ylabel("Log Price")
56 axes[2].plot(log_returns, color="darkgreen",
57              linewidth=0.6, alpha=0.8)
58 axes[2].axhline(0, color="red", linewidth=1,
59               linestyle="--")
60 axes[2].set_title("Log Returns = delta log(Price)
61                  (STATIONARY)")
62 axes[2].set_ylabel("Log Return")
63 plt.tight_layout(); plt.show()
64
65 #          PART D -- KPSS Confirmation [3 marks]
66
67 # ADF H0: Series has a unit root (NON-STATIONARY) --
68 # reject to claim stationarity
69 # KPSS H0: Series IS STATIONARY -- reject (stat > CV) to
70 # claim non-stationarity
71 kpss_result = kpss(log_returns, regression="c",
72                   nlags="auto")
73 kpss_rejects = kpss_result[0] > kpss_result[3]["5%"]
74 print(f"\nPART D -- KPSS Test: MRF Log Returns")
75 print(f"    KPSS Statistic : {kpss_result[0]:.4f}")
76 print(f"    KPSS 5% CV      : {kpss_result[3]['5%']:.4f}")
77 print(f"    KPSS rejects H0: {kpss_rejects}")
78 print(f"    ADF rejects H0 : {adf_ret[1] < 0.05}")
79 if (adf_ret[1] < 0.05) and (not kpss_rejects):
80     print("\n    BOTH ADF and KPSS AGREE: Log returns are
81         STATIONARY")
82     print("    Strong evidence: d=1 is the correct
83         differencing order for MRF prices.")
84 else:
85     print("\n    Tests DISAGREE -- investigate structural
86         breaks in MRF data")

```

② Plain English

This problem walks through the full stationarity investigation: raw prices fail (as expected), log transform doesn't fix stationarity (it fixes variance but not trend), first-differencing does fix it, and KPSS confirms from the opposite direction. Together, ADF + KPSS = bulletproof evidence that $d = 1$ is correct.

③ Common Mistake — Part B

Many students think log transformation achieves stationarity because the chart “looks less steep.” It doesn't — if the mean still trends, the series is still non-stationary. Log transform is about variance stabilisation, not mean stabilisation. Mean stabilisation requires differencing.

④ Real World Link

Every serious financial econometrics paper on MRF or any Indian large-cap reports exactly this analysis — ADF + KPSS on levels and returns — as Table 1 of their empirical section. Knowing how to produce and interpret this table is a core professional skill for any data scientist working with financial time series.

M2.036 Exam Style — ARIMA Model Identification and Selection EXAM

Marks: 20 | Time: 35 minutes | Dataset: MRF.NS

Part A — ACF/PACF Identification

[5 marks]

Compute MRF log returns. Plot ACF and PACF (40 lags each). Based on visual inspection, propose candidate ARIMA orders for the returns series. Justify your (p, q) choices in code comments.

Part B — Fit Three Candidate Models

[5 marks]

Fit ARIMA(1,0,0), ARIMA(0,0,1), and ARIMA(1,0,1) to MRF log returns. For each model, print: order, AIC, BIC, and whether coefficients are statistically significant (p-value < 0.05).

Part C — Model Selection and Justification

[5 marks]

Select the best model using AIC. Justify in a comment block why AIC favours the selected model. State whether BIC agrees or disagrees, and why they might differ.

Part D — Residual Validation

[5 marks]

On the best model's residuals: run Ljung-Box (20 lags), plot ACF of residuals, and plot a histogram. Write a 3-sentence verdict on whether the model is correctly specified.

Solution

① Complete Solution

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
```

```

5 import scipy.stats as stats
6 import warnings
7 warnings.filterwarnings('ignore')
8 from statsmodels.tsa.arima.model import ARIMA
9 from statsmodels.graphics.tsaplots import plot_acf,
    plot_pacf
10 from statsmodels.stats.diagnostic import acorr_ljungbox
11
12 mrf = yf.download("MRF.NS", start="2019-01-01",
    end="2024-12-31")
13 returns = np.log(mrf["Close"]).diff().dropna()
14
15 # PART A -- ACF/PACF Identification [5 marks]
16
17 fig, axes = plt.subplots(1, 2, figsize=(16, 5))
18 plot_acf(returns, lags=40, alpha=0.05, ax=axes[0],
    zero=False)
19 axes[0].set_title("ACF -- MRF Log Returns (MA order: q)")
20 plot_pacf(returns, lags=40, alpha=0.05, ax=axes[1],
    zero=False)
21 axes[1].set_title("PACF -- MRF Log Returns (AR order: p)")
22 fig.suptitle("PART A -- ACF and PACF: MRF Log Returns",
    fontsize=12)
23 plt.tight_layout(); plt.show()
24 # Rule: PACF cuts off at lag p -> AR(p) / ACF cuts off at
    lag q -> MA(q)
25 # For MRF returns: most spikes within bands -> low-order
    process
26 # Candidates: ARIMA(1,0,0), ARIMA(0,0,1), ARIMA(1,0,1)
27
28 # PART B -- Fit Three Candidate Models [5 marks]
29
30 candidates = {
31     "ARIMA(1,0,0)": (1,0,0),
32     "ARIMA(0,0,1)": (0,0,1),
33     "ARIMA(1,0,1)": (1,0,1),
34 }
35 fits = {}
36 report = []
37 print("PART B -- Model Comparison"); print("="*65)
38 for name, order in candidates.items():
39     fit = ARIMA(returns, order=order).fit()
40     fits[name] = fit
41     pvals = fit.pvalues
42     sig = {k: ("Sig" if v < 0.05 else "Not sig") for k,
        v in pvals.items()}
43     report.append({"Model": name, "AIC": round(fit.aic,2),
        "BIC": round(fit.bic,2)})

```

```

42     print(f"\n{name}   AIC={fit.aic:.2f}
        BIC={fit.bic:.2f}")
43     for param, mark in sig.items():
44         print(f"       {mark}: {param}
        (p={pvals[param]:.4f})")
45 print("="*65)
46
47 #           PART C -- Model Selection [5 marks]
48
48 sorted_aic = sorted(report, key=lambda x: x["AIC"])
49 best_name  = sorted_aic[0]["Model"]
50 best_fit   = fits[best_name]
51 print(f"\nPART C -- Best by AIC: {best_name}
        (AIC={sorted_aic[0]['AIC']})")
52 # AIC penalises each parameter by 2; BIC penalises by
    log(n)
53 # For MRF returns  $n \sim 1250$ ,  $\log(1250) \sim 7.1 >> 2$ 
54 # -> BIC applies much stronger penalty -> may prefer
    simpler model
55 sorted_bic = sorted(report, key=lambda x: x["BIC"])
56 bic_winner = sorted_bic[0]["Model"]
57 print(f"Best by BIC: {bic_winner}
        (BIC={sorted_bic[0]['BIC']})")
58 print(f"AIC and BIC agree: {best_name == bic_winner}")
59
60 #           PART D -- Residual Validation [5 marks]
61
61 resid = best_fit.resid.dropna()
62 fig, axes = plt.subplots(1, 3, figsize=(18, 4))
63 fig.suptitle(f"PART D -- Residual Diagnostics:
        {best_name}", fontsize=12)
64 plot_acf(resid, lags=30, alpha=0.05, ax=axes[0],
        zero=False)
65 axes[0].set_title("ACF of Residuals")
66 axes[1].hist(resid, bins=60, density=True,
        color="steelblue", alpha=0.6)
67 x = np.linspace(resid.min(), resid.max(), 300)
68 axes[1].plot(x, stats.norm.pdf(x, resid.mean(),
        resid.std()), "r-", lw=2)
69 axes[1].set_title("Residual Histogram")
70 stats.probplot(resid, dist="norm", plot=axes[2])
71 axes[2].set_title("Q-Q Plot")
72 plt.tight_layout(); plt.show()
73
74 lb     = acorr_ljungbox(resid, lags=20, return_df=True)
75 n_sig  = (lb["lb_pvalue"] < 0.05).sum()
76 min_p  = lb["lb_pvalue"].min()
77 print(f"\nLjung-Box: {n_sig}/20 significant lags | Min
        p-value: {min_p:.4f}")

```

```

78 # 3-SENTENCE VERDICT:
79 verdict = [
80     f"1. Ljung-Box: {n_sig} significant lags -- residuals
      are "
81     f"{'consistent with white noise' if n_sig==0 else 'not
      white noise'}.",
82     "2. Q-Q plot shows heavier tails than normal -- GARCH
      effects likely present.",
83     "3. ARIMA captures mean dynamics; pair with GARCH
      (Module 4) for complete model.",
84 ]
85 print("\n VERDICT:")
86 for line in verdict: print(f" {line}")

```

④ Real World Link

This complete workflow — ACF/PACF identification → multi-model comparison → AIC selection → residual validation → GARCH recommendation — is exactly the analytical process a data scientist would follow when building a return model for MRF for portfolio risk management or algorithmic trading.

M2.037 Exam Style — SARIMA for MRF Monthly Data EXAM

Marks: 20 | Time: 35 minutes | Dataset: MRF.NS Monthly

Part A — Seasonal ACF Evidence [5 marks]

Download MRF, resample to monthly. Plot ACF of the monthly log-differenced series (40 lags). Identify whether spikes at lags 12, 24 suggest seasonal structure. Write conclusion in comments.

Part B — SARIMA Specification and Fitting [5 marks]

Fit SARIMA(1,1,1)(1,1,0)[12] to the monthly series. Print model summary. Report seasonal AR coefficient value and p-value.

Part C — In-Sample Fit Plot [5 marks]

Plot observed monthly prices vs model fitted values (in-sample). Compute and print Root Mean Squared Error (RMSE) of in-sample fit.

Part D — 12-Month Ahead Forecast [5 marks]

Generate a 12-month ahead forecast with 95% confidence intervals. Plot the forecast extending from the historical series. Annotate the chart with the RMSE from Part C.

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np

```

```

4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import warnings
7 warnings.filterwarnings('ignore')
8 from statsmodels.graphics.tsaplots import plot_acf
9 from statsmodels.tsa.statespace.sarimax import SARIMAX
10
11 mrf = yf.download("MRF.NS", start="2019-01-01",
12                 end="2024-12-31")
13 close = mrf["Close"].dropna()
14 monthly = close.resample("ME").last()
15
16 # PART A -- Seasonal ACF Evidence [5 marks]
17
18 monthly_diff = monthly.diff(12).diff().dropna() #
19 # seasonal then regular diff
20 fig, ax = plt.subplots(figsize=(14, 5))
21 plot_acf(monthly_diff, lags=40, alpha=0.05, ax=ax,
22         zero=False)
23 ax.set_title("PART A -- ACF: Monthly MRF (seasonally +
24         regularly differenced)")
25 plt.tight_layout(); plt.show()
26 # Significant spike at lag 12 -> annual seasonal
27 # correlation present
28 # Significant spike at lag 24 -> harmonic (2nd seasonal
29 # cycle)
30 # Absence of spikes at s=12,24 -> seasonal differencing
31 # was sufficient
32
33 # PART B -- SARIMA Fitting [5 marks]
34
35 model = SARIMAX(monthly,
36                 order=(1, 1, 1),
37                 seasonal_order=(1, 1, 0, 12),
38                 enforce_stationarity=False,
39                 enforce_invertibility=False)
40 fit = model.fit(dispatch=False)
41 print("PART B -- SARIMA(1,1,1)(1,1,0)[12] Summary")
42 print(fit.summary())
43
44 # Report seasonal AR coefficient (at lag 12)
45 try:
46     sar_coef = fit.params["ar.S.L12"]
47     sar_pval = fit.pvalues["ar.S.L12"]
48 except KeyError:
49     sar_coef = fit.params.iloc[-3]
50     sar_pval = fit.pvalues.iloc[-3]
51 print(f"\nSeasonal AR (lag 12): coef={sar_coef:.4f},
52       p={sar_pval:.4f}")

```



```

43 print(f"Significant at 5%? {'Yes' if sar_pval < 0.05 else
    'No'}")
44
45 #          PART C -- In-Sample Fit and RMSE [5 marks]
46 fitted_vals = fit.fittedvalues
47 observed    = monthly[fitted_vals.index]
48 rmse = np.sqrt(((observed - fitted_vals)**2).mean())
49 print(f"\nPART C -- In-Sample RMSE: INR {rmse:.2f}")
50
51 fig, ax = plt.subplots(figsize=(14, 5))
52 ax.plot(observed, color="steelblue", linewidth=1.5,
    label="Observed MRF Price")
53 ax.plot(fitted_vals, color="red", linewidth=1.5,
    linestyle="--", label="SARIMA Fitted")
54 ax.set_title("PART C -- MRF Monthly: Observed vs SARIMA
    Fitted Values")
55 ax.set_ylabel("INR"); ax.legend(); plt.tight_layout();
    plt.show()
56
57 #          PART D -- 12-Month Forecast [5 marks]
58 fc_obj = fit.get_forecast(steps=12)
59 fc_mean = fc_obj.predicted_mean
60 fc_ci = fc_obj.conf_int(alpha=0.05)
61
62 fig, ax = plt.subplots(figsize=(14, 6))
63 ax.plot(monthly[-36:], color="steelblue", linewidth=1.5,
    label="Historical MRF")
64 ax.plot(fc_mean, color="red", linewidth=2, label="12-Month
    Forecast")
65 ax.fill_between(fc_ci.index, fc_ci.iloc[:,0],
    fc_ci.iloc[:,1],
66                 color="red", alpha=0.2, label="95%
    Confidence Interval")
67 ax.axvline(x=monthly.index[-1], color="black",
    linestyle="--", linewidth=1)
68 ax.annotate(f"In-sample RMSE: INR{rmse:.0f}",
69             xy=(0.02, 0.95), xycoords="axes fraction",
    fontsize=10, color="darkred",
70             bbox=dict(boxstyle="round,pad=0.3",
    facecolor="lightyellow"))
71 ax.set_title("PART D -- SARIMA(1,1,1)(1,1,0)[12]: 12-Month
    MRF Forecast")
72 ax.set_ylabel("INR"); ax.legend(); plt.tight_layout();
    plt.show()

```

④ Real World Link

SARIMA monthly forecasts for MRF are directly useful for inventory planning, revenue projection, and equity research (12-month price targets). The wide confidence intervals at 12 months are honest — they tell management to plan for a range of outcomes, not a single number.

M2.038 Exam Style — Model Comparison: ARIMA vs SARIMA vs Random Walk EXAM

Marks: 15 | Time: 30 minutes | Dataset: MRF.NS

Part A — Three Models

[6 marks]

On MRF monthly prices, fit: (1) Random Walk ARIMA(0,1,0), (2) ARIMA(1,1,1), (3) SARIMA(1,1,1)(1,1,0)[12]. Print AIC and BIC for all three in a formatted comparison table.

Part B — Walk-Forward Validation

[6 marks]

Use the last 12 months as a test set and the preceding months as training. Fit ARIMA(1,1,1) and SARIMA(1,1,1)(1,1,0)[12] on training data. Forecast 12 months. Compute test RMSE for both. Print results.

Part C — Final Recommendation

[3 marks]

Write a structured comment block (6 lines max) recommending which model to use for MRF monthly prices, citing AIC, BIC, and test RMSE as evidence.

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import warnings
7 warnings.filterwarnings('ignore')
8 from statsmodels.tsa.arima.model import ARIMA
9 from statsmodels.tsa.statespace.sarimax import SARIMAX
10
11 mrf = yf.download("MRF.NS", start="2019-01-01",
12                  end="2024-12-31")
13 monthly = mrf["Close"].dropna().resample("ME").last()
14 # PART A -- Three Models, AIC/BIC Table [6 marks]
15 models = {
16     "Random Walk ARIMA(0,1,0)": ARIMA(monthly,
17                                       order=(0,1,0)),
17     "ARIMA(1,1,1)": ARIMA(monthly,
18                           order=(1,1,1)),

```

```

18     "SARIMA(1,1,1)(1,1,0)[12]": SARIMAX(monthly,
19                                     order=(1,1,1),
20                                     seasonal_order=(1,1,0,12),
21                                     enforce_stationarity=False,
22                                     enforce_invertibility=False),
23 }
24 fits_a = {}; rows = []
25 for name, model in models.items():
26     fit = model.fit(dispatch=False)
27     fits_a[name] = fit
28     rows.append({"Model": name, "AIC": round(fit.aic,2),
29               "BIC": round(fit.bic,2)})
30
31 comparison_df =
32     pd.DataFrame(rows).sort_values("AIC").reset_index(drop=True)
33 print("PART A -- Model Comparison Table")
34 print("="*60)
35 print(comparison_df.to_string(index=False))
36 print("="*60)
37 print(f"\nBest AIC: {comparison_df.iloc[0]['Model']}")
38 print(f"Best BIC:
39     {comparison_df.sort_values('BIC').iloc[0]['Model']}")
40
41 #           PART B -- Walk-Forward Validation [6 marks]
42
43 train = monthly[:-12]
44 test  = monthly[-12:]
45
46 arima_tr  = ARIMA(train, order=(1,1,1)).fit(dispatch=False)
47 arima_fc  = arima_tr.forecast(steps=12)
48
49 sarima_tr = SARIMAX(train, order=(1,1,1),
50                     seasonal_order=(1,1,0,12),
51                     enforce_stationarity=False,
52                     enforce_invertibility=False).fit(dispatch=False)
53 sarima_fc = sarima_tr.forecast(steps=12)
54
55 rmse_arima = np.sqrt(((test.values -
56     arima_fc.values)**2).mean())
57 rmse_sarima = np.sqrt(((test.values -
58     sarima_fc.values)**2).mean())
59
60 print(f"\nPART B -- Walk-Forward Validation (held-out 12
61     months)")
62 print(f"    ARIMA(1,1,1)    Test RMSE : INR {rmse_arima:,.2f}")
63 print(f"    SARIMA(...)    Test RMSE : INR
64     {rmse_sarima:,.2f}")
65 print(f"    Winner (lower RMSE): {'ARIMA' if rmse_arima <
66     rmse_sarima else 'SARIMA'}")

```

```

55
56 fig, ax = plt.subplots(figsize=(14, 5))
57 ax.plot(monthly, color="steelblue", label="Historical",
58         linewidth=1.5)
59 ax.plot(test.index, arima_fc.values, color="red",
60         label=f"ARIMA (RMSE=INR{rmse_arima:,.0f})",
61         linewidth=2)
62 ax.plot(test.index, sarima_fc.values, color="green",
63         label=f"SARIMA (RMSE=INR{rmse_sarima:,.0f})",
64         linewidth=2)
65 ax.axvline(x=train.index[-1], color="gray", linestyle="--")
66 ax.legend(); ax.set_title("PART B -- Walk-Forward
67     Validation: ARIMA vs SARIMA")
68 plt.tight_layout(); plt.show()
69
70 # PART C -- Final Recommendation [3 marks]
71
72 # RECOMMENDATION COMMENT BLOCK:
73 # Model recommended: whichever has lower test RMSE
74 # (typically SARIMA for monthly)
75 #
76 # Evidence:
77 # 1. AIC: SARIMA typically wins (captures seasonal
78 #    structure -> better fit)
79 # 2. BIC: ARIMA may win (stronger parameter penalty;
80 #    SARIMA has more params)
81 # 3. Test RMSE: SARIMA typically wins for monthly data
82 #    with annual seasonality
83 #
84 # Reasoning: MRF's annual tyre demand cycle creates
85 #    genuine seasonal autocorrelation
86 #    at lag 12. SARIMA explicitly models this, producing
87 #    lower out-of-sample errors.
88 # Caveat: With ~60 monthly obs, SARIMA risks overfitting.
89 #    If RMSE difference < 5%,
90 #    prefer ARIMA(1,1,1) for parsimony.
91 winner = "SARIMA" if rmse_sarima < rmse_arima else "ARIMA"
92 print(f"\nPART C -- Recommendation: {winner}")
93 print(f"    ARIMA RMSE=INR{rmse_arima:,.0f} | SARIMA
94     RMSE=INR{rmse_sarima:,.0f}")

```

④ Real World Link

Walk-forward validation (time-series cross-validation) is the only honest way to assess a financial forecast model. Splitting MRF data into train/test chronologically — never randomly, since time order is fundamental — and comparing out-of-sample RMSE gives you the real-world evidence needed to choose between ARIMA and SARIMA.

M2.039 Exam Style — Complete Box-Jenkins Pipeline **EXAM****Marks: 20 | Time: 40 minutes**

Build a complete, documented ARIMA model for MRF log returns following the full Box-Jenkins methodology. Code must be executable top to bottom in a fresh Colab session.

- **Part A (4 marks):** Data download, log returns, ADF test with conclusion comment
- **Part B (4 marks):** ACF/PACF plots with order selection logic in comments
- **Part C (4 marks):** Fit 4 candidate models, AIC/BIC comparison table
- **Part D (4 marks):** Best model residual diagnostics (ACF, histogram, Ljung-Box verdict)
- **Part E (4 marks):** 30-day forecast plot with confidence intervals and interpretation

Solution**① Complete Solution**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, scipy.stats as spstats
4 import warnings; warnings.filterwarnings('ignore')
5 from statsmodels.tsa.arima.model import ARIMA
6 from statsmodels.tsa.stattools import adfuller
7 from statsmodels.graphics.tsaplots import plot_acf,
  plot_pacf
8 from statsmodels.stats.diagnostic import acorr_ljungbox
9
10 #          PART A -- Data, Returns, Stationarity [4 marks]
11 mrf      = yf.download("MRF.NS", start="2019-01-01",
12                       end="2024-12-31")
13 close    = mrf["Close"].dropna()
14 returns  = np.log(close).diff().dropna()
15 adf      = adfuller(returns)
16 print(f"PART A -- ADF on MRF Log Returns")
17 print(f"  n = {len(returns)} | ADF stat = {adf[0]:.4f}
18       | p = {adf[1]:.6f}")
19 print(f"  {'STATIONARY (d=0 for returns, d=1 for prices)'
20       | if adf[1]<0.05 else 'Non-stationary'}")
21
22 #          PART B -- ACF/PACF Order Selection [4 marks]
23
24 fig, axes = plt.subplots(1, 2, figsize=(16, 5))
25 plot_acf( returns, lags=40, alpha=0.05, ax=axes[0],
26          zero=False)
27 axes[0].set_title("ACF -- q identification\nLook for: last
28                  significant lag = q")

```

```

23 plot_pacf(returns, lags=40, alpha=0.05, ax=axes[1],
    zero=False)
24 axes[1].set_title("PACF -- p identification\nLook for:
    last significant lag = p")
25 fig.suptitle("PART B -- ACF and PACF: MRF Log Returns",
    fontsize=12)
26 plt.tight_layout(); plt.show()
27 # Most spikes within bands -> low-order process expected
28 # Try p,q in {0,1} systematically: ARMA(0,0), AR(1),
    MA(1), ARMA(1,1)
29
30 # PART C -- Four Candidate Models, AIC/BIC [4
    marks]
31 candidate_orders = [(0,0,0), (1,0,0), (0,0,1), (1,0,1)]
32 model_results = []; fitted_models = {}
33 for order in candidate_orders:
34     fit = ARIMA(returns, order=order).fit()
35     fitted_models[order] = fit
36     model_results.append({
37         "Model": f"ARIMA{order}", "Params":
            int(fit.df_model)+1,
38         "AIC": round(fit.aic,2), "BIC": round(fit.bic,2),
            "Log-Lik": round(fit.llf,2),
39     })
40 results_df =
    pd.DataFrame(model_results).sort_values("AIC").reset_index(drop=True)
41 print("\nPART C -- Candidate Model Comparison (sorted by
    AIC)")
42 print("="*62)
43 print(results_df.to_string(index=False))
44 print("="*62)
45 best_order = tuple(int(x) for x in
    results_df.iloc[0]["Model"]
46                     .replace("ARIMA","").strip("(")").split(",")
47 best_fit = fitted_models[best_order]
48 print(f"\nSelected: ARIMA{best_order}
    AIC={best_fit.aic:.2f} | BIC={best_fit.bic:.2f}")
49
50 # PART D -- Residual Diagnostics [4 marks]
51 resid = best_fit.resid.dropna()
52 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
53 fig.suptitle(f"PART D -- Residual Diagnostics:
    ARIMA{best_order}", fontsize=12)
54 plot_acf(resid, lags=30, alpha=0.05, ax=axes[0],
    zero=False)
55 axes[0].set_title("ACF of Residuals (want: all inside
    bands)")

```

```

56 axes[1].hist(resid, bins=60, density=True,
    color="steelblue", alpha=0.6)
57 x_n = np.linspace(resid.min(), resid.max(), 300)
58 axes[1].plot(x_n, spstats.norm.pdf(x_n, resid.mean(),
    resid.std()), "r-", lw=2)
59 axes[1].set_title("Residual Histogram")
60 spstats.probplot(resid, dist="norm", plot=axes[2])
61 axes[2].set_title("Q-Q Plot")
62 plt.tight_layout(); plt.show()
63
64 lb = acorr_ljungbox(resid, lags=20, return_df=True)
65 n_sig = (lb["lb_pvalue"] < 0.05).sum(); min_p =
    lb["lb_pvalue"].min()
66 print(f"\nLjung-Box: {n_sig}/20 lags significant | min
    p={min_p:.4f}")
67 verdict = "Well-specified -- white noise residuals" if
    n_sig==0 else \
68     "Residuals have autocorrelation -- try higher
        order"
69 print(f"Verdict: {verdict}")
70
71 # PART E -- 30-Day Forecast [4 marks]
72
73 fc_obj = best_fit.get_forecast(steps=30)
74 fc_mean = fc_obj.predicted_mean
75 fc_ci = fc_obj.conf_int(alpha=0.05)
76
77 fig, ax = plt.subplots(figsize=(14, 5))
78 ax.plot(range(120), returns[-120:].values,
    color="steelblue", linewidth=0.8,
    label="MRF Returns (last 120 days)")
79 x_fc = range(120, 150)
80 ax.plot(x_fc, fc_mean.values, color="red", linewidth=2,
    label="30-Day Forecast")
81 ax.fill_between(x_fc, fc_ci.iloc[:,0], fc_ci.iloc[:,1],
82     color="red", alpha=0.2, label="95%
        Confidence Interval")
83 ax.axhline(0, color="black", linewidth=0.8,
    linestyle="--", label="Zero (mean)")
84 ax.axvline(120, color="gray", linestyle="--", linewidth=1)
85 ax.set_title(f"PART E -- ARIMA{best_order}: MRF 30-Day
    Return Forecast")
86 ax.set_xlabel("Days"); ax.set_ylabel("Log Return")
87 ax.legend(fontsize=9); plt.tight_layout(); plt.show()
88
89 # FORECAST INTERPRETATION:
90 # Point forecast converges quickly to ~0 (long-run mean of
    returns) -- expected

```

```

91 # for ARIMA with small AR/MA coefficients. Wide confidence
    intervals reflect
92 # genuine uncertainty. ARIMA captures mean dynamics; pair
    with GARCH (Module 4)
93 # for volatility forecasting of MRF returns.
94 print(f"\nPART E -- Forecast Summary:")
95 print(f"    30-day range : [{fc_mean.min():.4f},
    {fc_mean.max():.4f}]")
96 print(f"    Final value  : {fc_mean.iloc[-1]:.6f} (converges
    to 0 -- expected)")
97 print(f"    CI width day 30:
    {(fc_ci.iloc[-1,1]-fc_ci.iloc[-1,0]):.4f}")

```

④ Real World Link

This complete workflow — ADF → ACF/PACF → AIC grid → residual diagnostics → forecast — is exactly the analytical process a data scientist follows when building a return model for MRF for portfolio risk management or algorithmic trading.

M2.040 Exam Style — Integrated Challenge EXAM

Marks: 25 | Time: 50 minutes | Dataset: MRF.NS

Part A (5 marks): Download MRF. Run ADF on raw price, log price, and log returns. Create a 3-row summary table: Series | ADF stat | p-value | Stationary?

Part B (5 marks): Fit ARIMA($p,0,q$) on returns for all $(p,q) \in \{0,1,2\}^2$. Create AIC heatmap (3×3 grid of coloured cells). Highlight the best cell.

Part C (5 marks): Fit best ARIMA from Part B. Full residual diagnostics. Write structured conclusion.

Part D (5 marks): Resample to monthly. Fit SARIMA(1,1,1)(1,1,0)[12]. Forecast 12 months. Plot.

Part E (5 marks): Write a 10-line professional analysis comment summarizing: data used, stationarity findings, model selected, residual quality, and forecast interpretation.

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, warnings
4 import scipy.stats as spstats
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.arima.model import ARIMA
7 from statsmodels.tsa.stattools import adfuller
8 from statsmodels.graphics.tsaplots import plot_acf
9 from statsmodels.stats.diagnostic import acorr_ljungbox
10 from statsmodels.tsa.statespace.sarimax import SARIMAX

```



```

11
12 mrf = yf.download("MRF.NS", start="2019-01-01",
13                 end="2024-12-31")
14 close = mrf["Close"].dropna()
15 # PART A -- ADF Summary Table [5 marks]
16 series_dict = {
17     "Raw Price" : close,
18     "Log Price" : np.log(close),
19     "Log Returns" : np.log(close).diff().dropna(),
20 }
21 adf_rows = []
22 for name, s in series_dict.items():
23     res = adfuller(s)
24     adf_rows.append({"Series": name, "ADF Stat":
25                     round(res[0],4),
26                     "p-value": round(res[1],6),
27                     "Stationary?": "YES" if res[1]<0.05
28                     else "NO"})
29 adf_table = pd.DataFrame(adf_rows)
30 print("PART A -- Stationarity Summary Table")
31 print("="*60); print(adf_table.to_string(index=False));
32 print("="*60)
33 # PART B -- AIC Heatmap [5 marks]
34 returns = np.log(close).diff().dropna()
35 orders = [0, 1, 2]
36 aic_grid = np.full((3,3), np.nan)
37 for i, p in enumerate(orders):
38     for j, q in enumerate(orders):
39         try:
40             aic_grid[i,j] = ARIMA(returns,
41                                   order=(p,0,q)).fit().aic
42         except: pass
43 best_idx = np.unravel_index(np.nanargmin(aic_grid),
44                             aic_grid.shape)
45 best_p, best_q = orders[best_idx[0]], orders[best_idx[1]]
46 print(f"\nPART B -- Best ARIMA(p,0,q): p={best_p},
47       q={best_q}, AIC={aic_grid[best_idx]:.2f}")
48
49 fig, ax = plt.subplots(figsize=(7,5))
50 im = ax.imshow(aic_grid, cmap="RdYlGn_r", aspect="auto")
51 for i in range(3):
52     for j in range(3):
53         if not np.isnan(aic_grid[i,j]):

```

```

50         color = "white" if (i,j)==best_idx else
           "black"
51         weight = "bold" if (i,j)==best_idx else
           "normal"
52         ax.text(j, i, f"{aic_grid[i,j]:.1f}",
53                 ha="center", va="center", color=color,
54                 fontsize=11, fontweight=weight)
55     ax.set_xticks(range(3)); ax.set_xticklabels([f"q={q}" for
56         q in orders])
57     ax.set_yticks(range(3)); ax.set_yticklabels([f"p={p}" for
58         p in orders])
59     ax.set_title("PART B -- AIC Grid: ARIMA(p,0,q) on MRF Log
60         Returns\n(lower=better; best=bold/white)")
61     plt.colorbar(im, ax=ax, label="AIC"); plt.tight_layout();
62     plt.show()
63
64     # PART C -- Best Model Diagnostics [5 marks]
65
66     best_fit = ARIMA(returns, order=(best_p, 0, best_q)).fit()
67     resid = best_fit.resid.dropna()
68     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
69     fig.suptitle(f"PART C -- Diagnostics:
70         ARIMA({best_p},0,{best_q})", fontsize=12)
71     plot_acf(resid, lags=30, alpha=0.05, ax=axes[0],
72         zero=False)
73     axes[0].set_title("ACF of Residuals")
74     axes[1].hist(resid, bins=60, density=True,
75         color="steelblue", alpha=0.6)
76     x_n = np.linspace(resid.min(), resid.max(), 300)
77     axes[1].plot(x_n, spstats.norm.pdf(x_n, resid.mean(),
78         resid.std()), "r-", lw=2)
79     axes[1].set_title("Residual Histogram")
80     spstats.probplot(resid, dist="norm", plot=axes[2])
81     axes[2].set_title("Q-Q Plot"); plt.tight_layout();
82     plt.show()
83
84     lb = acorr_ljungbox(resid, lags=20, return_df=True)
85     n_sig = (lb["lb_pvalue"]<0.05).sum(); min_p =
86         lb["lb_pvalue"].min()
87     print(f"\nPART C -- Ljung-Box: {n_sig}/20 significant |
88         min p={min_p:.4f}")
89     print("Structured Conclusion:")
90     print(f"  1. Mean structure: {'Adequately captured' if
91         n_sig==0 else 'Residual autocorrelation remains'}")
92     print(f"  2. Distribution: Heavier-tailed than normal (Q-Q
93         curve at extremes)")
94     print(f"  3. Action: Pair ARIMA({best_p},0,{best_q}) with
95         GARCH(1,1) for complete model")
96

```

```

81 #          PART D -- SARIMA Monthly + 12-Month Forecast [5
    marks]
82 monthly = close.resample("ME").last()
83 sarima = SARIMAX(monthly, order=(1,1,1),
    seasonal_order=(1,1,0,12),
84                 enforce_stationarity=False,
    enforce_invertibility=False).fit(dispatch=False)
85 print(f"\nPART D -- SARIMA(1,1,1)(1,1,0)[12]:
    AIC={sarima.aic:.2f} | BIC={sarima.bic:.2f}")
86 print("NOTE: SARIMA uses monthly data (~72 obs); ARIMA
    used daily (~1250 obs)")
87 print("      -> AIC values NOT directly comparable across
    different data frequencies")
88
89 fc_s = sarima.get_forecast(steps=12)
90 fc_mean = fc_s.predicted_mean
91 fc_ci = fc_s.conf_int()
92 fig, ax = plt.subplots(figsize=(14, 5))
93 ax.plot(monthly, color="steelblue", linewidth=1.5,
    label="Historical MRF (Monthly)")
94 ax.plot(fc_mean, color="red", linewidth=2,
    label="12-Month SARIMA Forecast")
95 ax.fill_between(fc_ci.index, fc_ci.iloc[:,0],
    fc_ci.iloc[:,1],
96                 color="red", alpha=0.2, label="95% CI")
97 ax.axvline(monthly.index[-1], color="gray", linestyle="--")
98 ax.set_title("PART D -- SARIMA(1,1,1)(1,1,0)[12]: 12-Month
    MRF Price Forecast")
99 ax.set_ylabel("INR"); ax.legend(); plt.tight_layout();
    plt.show()
100
101 #          PART E -- Professional Analysis Summary [5 marks]
102
103 print(f"""
104 PART E -- MRF.NS Time Series Analysis: Professional Summary
105 =====
106 Data          : MRF.NS (NSE), 2019-01-01 to 2024-12-31, ~1250
    daily obs.
107
108                 Resampled to monthly (~72 obs) for seasonal
    modelling.
109
110 Stationarity: Raw and log prices non-stationary (ADF p >
    0.05 for both).
111
112                 First-differenced log prices (log returns)
    stationary (p < 0.001).
113
114                 ARIMA parameter: d=1 for price-level models;
    d=0 for return models.

```

```

112 Model (Daily): ARIMA({best_p},0,{best_q}) selected by AIC
      grid search over (p,q) in {{0,1,2}}^2.
113         Captures short-run autocorrelation in MRF log
      returns.
114         Residuals consistent with white noise on
      Ljung-Box (20 lags).
115
116 Model (Monthly): SARIMA(1,1,1)(1,1,0)[12] captures annual
      seasonality in MRF
117         monthly prices (tyre demand peaks in summer;
      drops during monsoon).
118
119 Residuals : Heavy-tailed (Q-Q deviation) and volatility
      clustering suggest
120         GARCH(1,1) should complement ARIMA for
      variance modelling (Module 4).
121
122 Forecast : 12-month SARIMA price forecast shows
      continued trend with seasonal
123         fluctuation. 95% CI widens substantially
      beyond 6 months -- reflects
124         genuine uncertainty and should not be treated
      as precise point estimates.
125
126 Recommendation: Use ARIMA({best_p},0,{best_q})+GARCH(1,1)
      for daily return/risk modelling.
127         Use SARIMA(1,1,1)(1,1,0)[12] for monthly
      price and revenue forecasting.
128 =====
129 """ .format(best_p=best_p, best_q=best_q))

```

④ Real World Link

Part E is the deliverable that matters most in a real job. The ability to write a concise, evidence-backed model summary — citing specific statistics, acknowledging limitations, and making a clear recommendation — separates a junior analyst from a senior one. In equity research, this paragraph becomes the “Quantitative Methodology” section of a research note on MRF sent to institutional investors.

Module 2 Complete

40 Problems — 10 Easy — 12 Medium — 12 Hard — 6 Exam

Range	Difficulty	Types
M2.001–M2.010	EASY	FIB, MCQ, FTE, PTO, SA
M2.011–M2.022	MEDIUM	FIB, FTE, GOR, MTF, MCQ, WFS
M2.023–M2.034	HARD	FIB, FTE, MCQ, WFS, PTO, SA
M2.035–M2.040	EXAM	Multi-part
Total	40 Problems	All 8 types

Problem	Type
M2.001–M2.002, M2.008, M2.011, M2.015, M2.019, M2.023, M2.028, M2.034	Fill in the Blank
M2.007, M2.012, M2.016, M2.022, M2.024, M2.029	Find the Error
M2.013	Get the Order Right
M2.014	Match the Following
M2.003, M2.004, M2.009, M2.017, M2.021, M2.025, M2.032, M2.033	Multiple Choice / PTO
M2.018, M2.020, M2.026, M2.031	Write from Scratch
M2.005, M2.027	Predict the Output
M2.006, M2.010, M2.030	Short Answer
M2.035–M2.040	Exam Style

Ready for **Module 3** — Forecasting & Model Selection: AIC/BIC, Diagnostics, Unit Root, Holt-Winters, Forecast Evaluation